

# Chương 1 Cơ sở lý thuyết

Chương này đưa ra cơ sở lý luận cho bốn mô hình được nghiên cứu ở phần sau. Ở phần đầu tiên, khái niệm biểu diễn văn bản và đặc trưng ngôn ngữ đặc thù được cụ thể hóa. Phần tiếp theo nghiên cứu nguyên lý áp dụng hai thuật toán học máy truyền thống LR và SVM. Phần cuối cùng đi sâu vào các kiến trúc học sâu từ LSTM tuần tự và BiLSTM hai chiều đến Transformer song song và đưa ra lời giải thích về vì sao PhoBERT với cơ chế self-attention sở hữu hiệu năng ưu việt trên bài toán phát hiện tin giả tiếng Việt.

## 1.1 Một số khái niệm cơ bản

### 1.1.1 Văn bản và biểu diễn văn bản

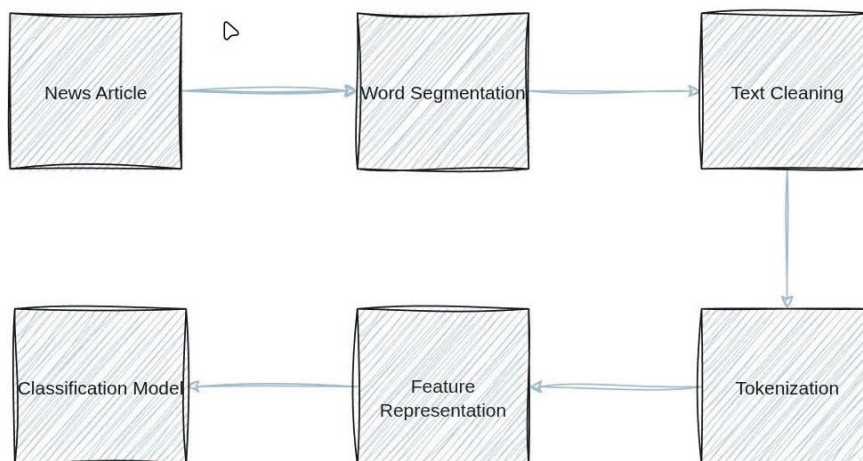
Mô hình học máy không thực hiện trực tiếp trên chuỗi ký tự mà cần biểu diễn văn bản dưới dạng số trong một không gian đặc trưng. Có hai trường phái biểu diễn đối lập nhau về bản chất là: Bag-of-Words và TF-IDF biểu diễn văn bản dưới dạng một vectơ thưa (*sparse vector*) trong không gian từ vựng, hoàn toàn bỏ qua thứ tự từ. Ngược lại, các phương pháp biểu diễn từ ngữ (*embedding*) biểu diễn từng từ ngữ dưới dạng một không gian đầy (*dense space*) với chiều thấp và thông tin ngữ nghĩa được mã hóa ngầm thông qua cấu trúc hình học của các vectơ. Nên chọn mô hình biểu diễn thưa hay đầy gắn liền với cấu trúc của từng mô hình là: LR và SVM thích hợp với mô hình biểu diễn thưa TF-IDF nhờ khả năng tối ưu tốt trong không gian chiều cao; BiLSTM và PhoBERT thích hợp với mô hình biểu diễn đầy nhờ khả năng khai thác ngữ cảnh toàn cục.

### 1.1.2 Đặc trưng văn bản và tiền xử lý

Tiền xử lý sẽ biến đổi văn bản thô thành đầu vào cho mô hình bằng cách chuẩn hóa chữ viết, loại bỏ ký tự nhiễu, tách từ và lọc từ dừng. Đối với tiếng Việt, *phân đoạn từ* (*word segmentation*) được xem là quan trọng hơn nhiều so với các phương pháp biến hình khác. Khác với tiếng Anh, các khoảng trắng trong tiếng Việt được sử dụng để phân tách *âm tiết* chứ không phải *từ*, do đó nhiều từ vựng thực chất được tạo thành từ hai hoặc nhiều âm tiết liền kề nhau (“kiến\_trúc”, “tiền\_huấn\_luyện”). Đây là bước quan trọng không thể bỏ qua vì nếu bỏ qua, dữ liệu sẽ bị phân mảnh nghiêm trọng,

làm giảm chất lượng biểu diễn vectơ TF-IDF và dẫn đến sự không tương thích giữa dữ liệu đầu vào với từ điển tiền huấn luyện của PhoBERT.

### Test preprocessing pipeline



Hình 1.1: Các bước tiền xử lý văn bản trước khi đưa vào mô hình học máy.

### 1.1.3 Bài toán phân loại văn bản

Phát hiện tin giả về bản chất là bài toán phân loại nhị phân trong đó mỗi văn bản được gán nhãn “thật” hoặc “giả”. Mô hình cần học một hàm ánh xạ từ không gian đặc trưng của văn bản sang tập nhãn, sao cho ranh giới quyết định phân tách tốt nhất hai lớp trên dữ liệu chưa quan sát.

## 1.2 Xử lý ngôn ngữ tự nhiên tiếng Việt

### 1.2.1 Đặc điểm ngôn ngữ tiếng Việt

Tiếng Việt là ngôn ngữ đơn lập, không biến hình, với các đặc điểm khác biệt so với tiếng Anh và nhiều ngôn ngữ châu Âu:

- **Không có khoảng cách giữa các âm tiết:** Ranh giới từ và âm tiết trong tiếng Việt không trùng nhau. Ví dụ, “thành phố” là một từ nhưng được viết thành hai âm tiết riêng biệt.
- **Từ ghép và từ láy:** Nhiều khái niệm được biểu đạt bằng tổ hợp nhiều âm tiết, ví dụ từ ghép “học\_sinh”, “kinh\_tế”, hay từ láy “lung\_linh”.

- **Thanh điệu:** Tiếng Việt có 6 thanh điệu, ảnh hưởng quan trọng đến ngữ nghĩa nhưng không ảnh hưởng đến cấu trúc hình thái học.
- **Ngôn ngữ cô lập:** Quan hệ ngữ pháp được biểu đạt qua trật tự từ và các hư từ, không qua biến đổi hình thái.

Những đặc điểm này của tiếng Việt đòi hỏi bước tiền xử lý đặc thù – đặc biệt là *phân đoạn từ* – trước khi được dùng trong các mô hình học máy và học sâu.

### 1.2.2 Phân đoạn từ (Word Segmentation)

Phân đoạn từ là bước xác định ranh giới các từ trong câu. Đối với tiếng Việt hiện nay, các mô hình xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP), đặc biệt là PhoBERT, được huấn luyện trên dữ liệu đã được phân đoạn.

**Ví dụ:**

*Trước:* “Thành phố Hồ Chí Minh là trung tâm kinh tế lớn nhất Việt Nam.”

*Sau:* “Thành\_phố Hồ\_Chí\_Minh là trung\_tâm kinh\_tế lớn nhất Việt\_Nam.”

Trong nghiên cứu này, nhóm sử dụng **VnCoreNLP RDRSegmenter** [?] thông qua thư viện `py_vncorenlp`. Đây là bộ phân đoạn cùng với bộ tokenizer mà mô hình PhoBERT được tiền huấn luyện, do đó đảm bảo được sự nhất quán về từ vựng

### 1.2.3 Biểu diễn văn bản số

Để các mô hình có thể học được, dữ liệu cần phải được chuyển đổi sang dạng số. Nghiên cứu này sử dụng ba phương pháp:

1. **TF-IDF:** Vectơ đặc trưng dựa trên tần suất từ, phương pháp này phù hợp cho LR và SVM.
2. **Nhúng từ (Word Embedding):** Vectơ dày đặc học được [?], khởi tạo từ FastText tiếng Việt [?], phù hợp cho BiLSTM.
3. **Mã hóa cặp byte (Byte-Pair Encoding – BPE):** Tách từ thành các đơn vị con, phương pháp này được dùng cho PhoBERT.

## 1.3 Học máy

### 1.3.1 Định nghĩa

Với sự phát triển của khoa học công nghệ, các ứng dụng tự động ngày càng trở nên phổ biến hơn trong đời sống chúng ta, một phần là nhờ vào các thuật toán học

máy. Học máy (Machine Learning – ML) là một nhánh của lĩnh vực trí tuệ nhân tạo (Artificial Intelligence – AI), chuyên nghiên cứu và phát minh các thuật toán cho phép máy tính có khả năng tự vận hành một công việc nhất định bằng các bộ dữ liệu đã có mà không cần can thiệp nhiều từ lập trình viên. Các thuật toán máy học thường được chia thành 2 loại chính, đó là:

Học có giám sát (Supervised Learning) gồm các thuật toán giúp máy tính xây dựng một hàm dựa trên dữ liệu có sẵn và đã được gán nhãn để có thể xác định nhãn của một dữ liệu mới.

Học không giám sát (Unsupervised Learning) gồm các thuật toán cho phép máy tính học từ các dữ liệu có sẵn nhưng chưa được gán nhãn. Từ đó, phân các dữ liệu này thành các cụm theo yêu cầu.

### **1.3.2 Một số thuật toán học máy tiêu biểu**

#### **1.3.2.1 Hồi quy Logistic (Logistic Regression – LR)**

Hồi quy Logistic (Logistic Regression) [?] là một thuật toán học máy có giám sát được sử dụng phổ biến trong các bài toán phân lớp nhị phân. Thuật toán này mô hình hóa mối quan hệ giữa các đặc trưng đầu vào và xác suất thuộc về một lớp thông qua một hàm logistic (sigmoid). Cụ thể, dữ liệu huấn luyện được biểu diễn dưới dạng các vector đặc trưng trong không gian nhiều chiều. Thuật toán sẽ học các tham số của mô hình sao cho xác suất dự đoán phù hợp nhất với nhãn thực tế của dữ liệu huấn luyện.

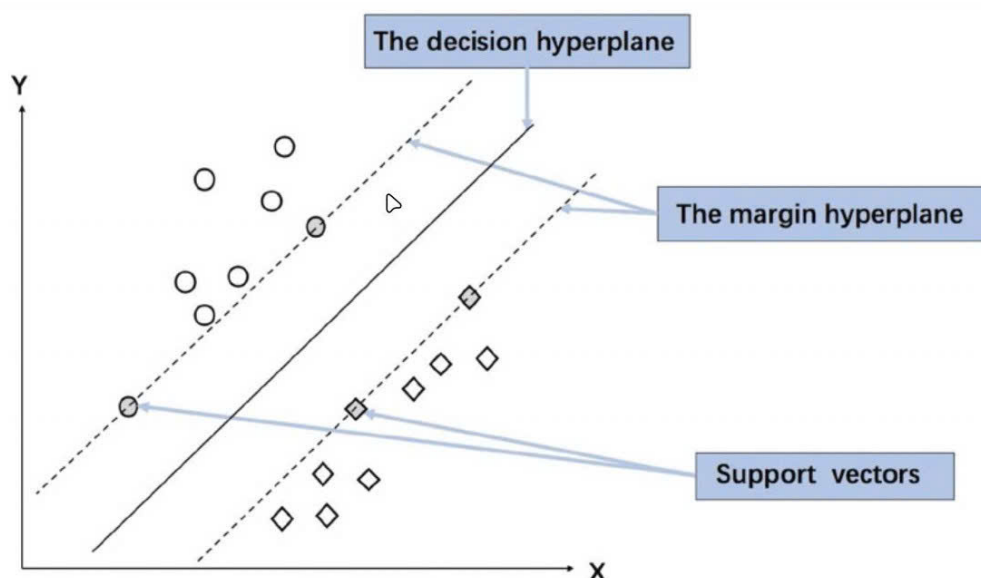
Khi có một điểm dữ liệu mới (chưa có nhãn), mô hình sẽ tính toán giá trị tuyến tính của điểm đó dựa trên các trọng số đã học, sau đó đưa kết quả qua hàm sigmoid để thu được một giá trị xác suất trong khoảng từ 0 đến 1. Dựa trên một ngưỡng phân loại xác định trước (thường là 0.5), điểm dữ liệu mới sẽ được gán vào một trong hai lớp tương ứng. Nếu xác suất dự đoán lớn hơn hoặc bằng ngưỡng, điểm dữ liệu sẽ được gán vào lớp thứ nhất; ngược lại, nó sẽ được gán vào lớp còn lại. Nhờ cơ chế này, Logistic Regression cho phép thực hiện phân lớp một cách hiệu quả và dễ diễn giải trong nhiều bài toán thực tế.

#### **1.3.2.2 Máy vectơ hỗ trợ (Support Vector Machine – SVM)**

Máy vectơ hỗ trợ (Support Vector Machine – SVM) [?] là một thuật toán học máy có giám sát được sử dụng phổ biến trong các bài toán phân lớp nhị phân. Đối với bài toán phát hiện tin giả, SVM được sử dụng để phân loại văn bản thành hai lớp là “thật” và “giả”. Thuật toán hoạt động dựa trên việc tìm một siêu phẳng (hyperplane) trong không gian đặc trưng sao cho có thể phân tách hai lớp dữ liệu một cách tối ưu.

Cụ thể, các dữ liệu huấn luyện sau khi được biểu diễn dưới dạng vector đặc trưng (ví dụ như TF-IDF) sẽ được ánh xạ vào không gian nhiều chiều. Trong không gian này, SVM tìm một siêu phẳng sao cho khoảng cách từ siêu phẳng đến các điểm gần nhất của mỗi lớp là lớn nhất. Khoảng cách này được gọi là biên (margin). Những điểm dữ liệu nằm gần siêu phẳng nhất và quyết định vị trí của nó được gọi là các vector hỗ trợ (support vectors).

Khi đã xác định được siêu phẳng tối ưu, với một điểm dữ liệu mới, mô hình sẽ xác định vị trí của điểm đó so với siêu phẳng để gán nhãn tương ứng. Nếu điểm dữ liệu nằm về một phía của siêu phẳng, nó sẽ được gán vào lớp thứ nhất; nếu nằm về phía còn lại, nó sẽ được gán vào lớp thứ hai. Nhờ cơ chế tối đa hóa biên phân tách, SVM thường đạt hiệu quả cao trong các bài toán phân loại văn bản nhị phân như phát hiện tin giả.



Hình 1.2: Minh họa thuật toán nhị phân SVM (phỏng theo [?])

### 1.3.3 Biểu diễn đặc trưng cơ bản

#### 1.3.3.1 Tần suất từ – Tần suất tài liệu nghịch đảo (TF-IDF)

TF-IDF (Term Frequency - Inverse Document Frequency) [?] là một trong những cách biểu diễn văn bản phổ biến trong các bài toán xử lý ngôn ngữ tự nhiên và phân loại văn bản. Phương pháp này đánh giá tính quan trọng của một từ trong một tài liệu nào đó theo tần suất xuất hiện của từ đó trong tài liệu và tính phổ biến của từ đó trong toàn bộ tập dữ liệu.

Term Frequency (TF) mô tả tần suất xuất hiện của một từ  $t$  trong một tài liệu  $d$ . Để giảm thiên lệch do chiều dài tài liệu, TF thường được chuẩn hóa theo tổng từ trong tài liệu:

$$TF(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Trong đó,  $f_{t,d}$  là số lần lặp lại của từ  $t$  trong văn bản  $d$ .

Inverse Document Frequency (IDF) được sử dụng để giảm trọng số của từ có tần suất lặp lại nhiều trong nhiều văn bản và tăng trọng số của từ có tính phân biệt cao. Cụ thể, công thức tính IDF như sau:

$$IDF(t) = \log \left( \frac{N}{df_t} \right)$$

trong đó  $N$  là tổng số tài liệu trong tập dữ liệu và  $df_t$  là số tài liệu chứa từ  $t$ .

Giá trị TF-IDF của một từ  $t$  trong tài liệu  $d$  được xác định bằng tích của hai thành phần trên:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Kết quả đạt được là một trọng số thực đại diện cho độ quan trọng tương đối của từng từ trong tài liệu đó. Sau khi tính toán cho toàn bộ từ vựng, mỗi tài liệu sẽ được biểu diễn dưới hình thức một vector đặc trưng với độ dài bằng tổng số từ trong từ điển. Các vector này thường là thưa (sparse) và thường dùng như một đầu vào cho các mô hình học máy truyền thống như Hồi quy Logistic (Logistic Regression) và Máy vectơ hỗ trợ (Support Vector Machine) cho các ứng dụng phân loại văn bản, như phát hiện thông tin giả.

## 1.4 Học sâu

### 1.4.1 Định nghĩa

Học sâu (Deep Learning) là một phương pháp trong học máy sử dụng các mô hình mạng nơ-ron nhân tạo có nhiều lớp nơ-ron ẩn nhằm giúp các hệ thống có khả năng học các đặc trưng phức tạp từ dữ liệu đầu vào. So với các phương pháp học máy truyền thống, trong đó các đặc trưng được tạo ra một cách thủ công, học sâu có khả năng tự động trích xuất các đặc trưng từ dữ liệu thông qua quá trình huấn luyện trên dữ liệu lớn.

Mỗi mô hình học sâu được xây dựng bằng các lớp nơ-ron được kết nối với nhau theo từng tầng. Dữ liệu đầu vào được truyền thông qua các lớp nơ-ron này theo cơ chế lan truyền tiến (feedforward), trong đó dữ liệu được xử lý tại từng lớp bằng các phép biến đổi tuyến tính kết hợp với hàm kích hoạt phi tuyến nhằm đạt được biểu diễn tại

mức trừu tượng cao. Trong quá trình huấn luyện, các tham số của mô hình được điều chỉnh thông qua thuật toán lan truyền ngược (backpropagation) [?] nhằm đạt được hàm mất mát tối thiểu.

Bên cạnh đó, nhờ có tính học nhiều tầng và khả năng học biểu diễn tự động, học sâu đã đạt được hiệu quả cao trong nhiều lĩnh vực như thị giác máy tính, xử lý ngôn ngữ tự nhiên, nhận dạng tiếng nói,... Đối với bài toán phát hiện tin giả, các mô hình học sâu như BiLSTM và PhoBERT có khả năng khai thác thông tin ngữ cảnh và mối quan hệ giữa các từ một cách hiệu quả hơn so với các phương pháp truyền thống.

### 1.4.2 Mạng nơ-ron nhân tạo (Artificial Neural Network)

Dựa trên cơ chế hoạt động của neuron trong cơ thể, các nhà khoa học đã xây dựng mô hình mạng nơ-ron nhân tạo nhằm mô phỏng hoạt động xử lý thông tin của bộ não người. Trong cơ thể, neuron là đơn vị cấu trúc cơ bản của hệ thần kinh. Mô hình neuron cơ bản gồm thân tế bào, sợi nhánh, và sợi trục. Thân tế bào neuron là nơi chứa nhân tế bào, sợi nhánh là nơi neuron nhận tín hiệu, và sợi trục là nơi neuron gửi tín hiệu cho neuron khác. Khi neuron nhận tín hiệu thông qua sợi nhánh, neuron sẽ thực hiện hoạt động xử lý và tổng hợp tín hiệu. Nếu tín hiệu sau khi xử lý lớn hơn ngưỡng nào đó, neuron sẽ gửi tín hiệu cho neuron khác thông qua sợi trục. Mô hình hoạt động của neuron đã được mô hình hóa trong mạng nơ-ron nhân tạo thông qua một hàm tính, trong đó nhiều tín hiệu đầu vào sẽ tạo nên tín hiệu đầu ra duy nhất. Vai trò của ngưỡng kích hoạt được mô phỏng thông qua hàm kích hoạt (Activation Function), quyết định xem neuron có phát tín hiệu hay không.

Mạng nơ-ron nhân tạo là một tập hợp các neuron nhân tạo được tổ chức thành nhiều lớp, bao gồm lớp đầu vào, một hoặc nhiều lớp ẩn và lớp đầu ra. Mỗi lớp ẩn có thể bao gồm một hoặc nhiều neuron nhân tạo. Các neuron trong một lớp được kết nối với các neuron ở lớp kế tiếp thông qua các trọng số, và không tồn tại kết nối giữa các neuron trong cùng một lớp.

Trong quá trình huấn luyện, mạng nơ-ron nhân tạo thực hiện cơ chế lan truyền tiến (Feedforward), trong đó dữ liệu đầu vào được truyền qua từng lớp và được biến đổi thông qua các trọng số tương ứng để tạo ra kết quả dự đoán ở đầu ra. Để đánh giá độ chính xác của dự đoán, một hàm mất mát (Loss Function) được sử dụng nhằm tính toán sai số giữa giá trị dự đoán và nhãn thực tế. Dựa trên sai số này, các tham số của mô hình sẽ được cập nhật thông qua thuật toán lan truyền ngược (Backpropagation) nhằm giảm thiểu sai số. Quá trình này được lặp lại nhiều lần cho đến khi mô hình đạt được trạng thái hội tụ.

Trong các kiến trúc mạng nơ-ron hiện đại, nhiều mô hình đã được phát triển nhằm xử lý các loại dữ liệu khác nhau. Đối với bài toán xử lý văn bản, các kiến trúc mạng

ơ-nơ hồi quy (Recurrent Neural Network – RNN) và các mô hình dựa trên Transformer được sử dụng phổ biến.

#### 1.4.2.1 Một số thành phần cần chú ý

Hàm kích hoạt (Activation Function) là một trong những thành phần quan trọng trong mô hình mạng ơ-nơ nhân tạo, có ý nghĩa quyết định đến việc liệu một tín hiệu đã được tổng hợp tại một ơ-nơ có được truyền đi đến ơ-nơ trong lớp kế tiếp hay không. Sau khi thực hiện quá trình tính toán các giá trị đầu vào được nhân với các trọng số tương ứng rồi cộng lại tại từng ơ-nơ trong mạng ơ-nơ nhân tạo, ta sử dụng hàm kích hoạt để tạo tính phi tuyến trong mô hình mạng ơ-nơ. Bởi vậy mà mô hình mạng ơ-nơ có khả năng học được các mối quan hệ trong dữ liệu phức tạp. Một số hàm kích hoạt được sử dụng trong mạng ơ-nơ nhân tạo là Sigmoid, Tanh, ReLU. Đối với các bài toán về phân loại dữ liệu, đặc biệt là bài toán phân loại nhị phân, hàm Sigmoid được sử dụng trong lớp output để dự đoán ra kết quả xác suất trong một khoảng giá trị từ 0 đến 1, trong khi hàm Softmax được sử dụng khi cần phân phối xác suất trên nhiều lớp.

Để xác định được độ chính xác của mô hình trong quá trình huấn luyện, người ta sử dụng hàm mất mát (Loss Function) với mục đích đo lường sai số giữa dự đoán và nhãn. Trong bài toán phân loại văn bản, hàm mất mát thường được sử dụng là hàm Cross-Entropy do có khả năng đo được sự chênh lệch giữa phân phối xác suất dự đoán và phân phối xác suất thực tế. Giá trị của hàm mất mát càng nhỏ thì mô hình càng đưa ra dự đoán gần với nhãn đúng.

Quá trình huấn luyện mạng ơ-nơ nhân tạo diễn ra qua hai quá trình chính: lan truyền tiến và lan truyền ngược. Trong quá trình lan truyền tiến, dữ liệu đầu vào được truyền qua từng lớp của mạng ơ-nơ nhân tạo. Trong quá trình truyền dữ liệu qua từng lớp của mạng ơ-nơ nhân tạo, các phép biến đổi tuyến tính và phi tuyến được thực hiện. Sau khi tính được kết quả của hàm mất mát, thuật toán lan truyền ngược được thực hiện với mục đích tính đạo hàm của sai số với từng tham số của mạng ơ-nơ nhân tạo. Các tham số của mạng ơ-nơ nhân tạo được cập nhật với thuật toán tối ưu hóa phổ biến nhất là thuật toán Gradient Descent với mục đích giảm thiểu sai số.

Trong quá trình huấn luyện các mạng nhiều tầng, có thể xuất hiện các hiện tượng như mất mát hoặc bùng nổ gradient khi giá trị đạo hàm trở nên quá nhỏ hoặc quá lớn. Những vấn đề này ảnh hưởng đến tốc độ và độ ổn định của quá trình học. Các kiến trúc hiện đại như LSTM và Transformer được thiết kế nhằm hạn chế những hạn chế này, giúp mô hình học được các quan hệ dài hạn trong dữ liệu chuỗi, đặc biệt hiệu quả trong các bài toán xử lý ngôn ngữ tự nhiên như phát hiện tin giả.



### 1.4.3 Mạng nơ-ron hồi quy (Recurrent Neural Network – RNN)

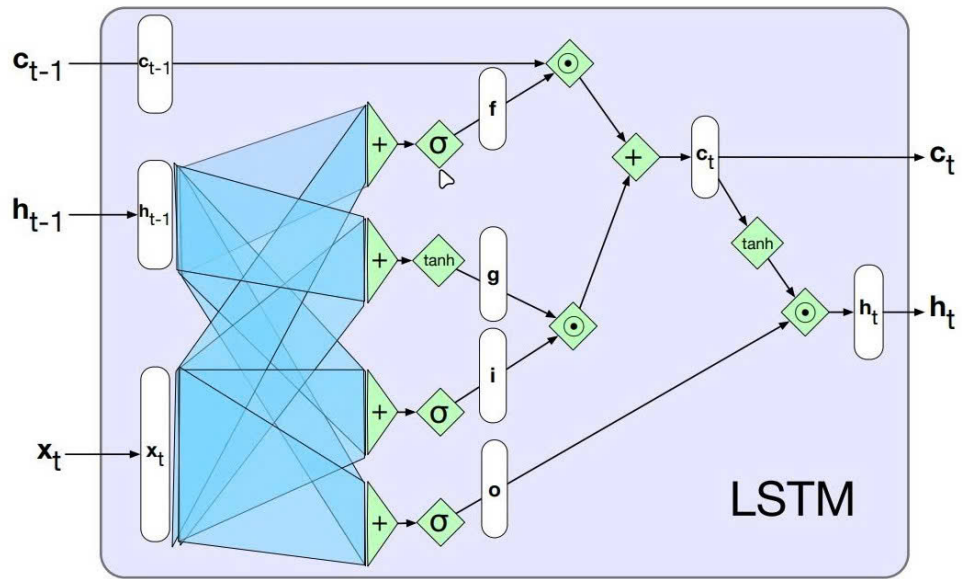
RNN là một mô hình học sâu được xây dựng nhằm xử lý dữ liệu tuần tự thông qua việc giữ lại thông tin từ các trạng thái trước đó [?]. Nhờ đó, RNN có khả năng nhớ và sử dụng các mối liên hệ giữa các phần tử trong dữ liệu tuần tự để tạo ra các kết quả có ngữ cảnh và ý nghĩa hơn.

Dữ liệu tuần tự có thể là văn bản, dữ liệu thời gian, hoặc các tín hiệu âm thanh trong đó các phần tử có sự liên kết với nhau dựa trên các ngữ nghĩa hoặc các quy luật xác định. So với các mạng nơ-ron truyền thống được sử dụng để xử lý các dữ liệu đầu vào độc lập với nhau, RNN được trang bị một vòng lặp nội tại để truyền thông tin trong từng bước thời gian.

Tuy nhiên, mặc dù có khả năng ghi nhớ thông tin thông qua các bước thời gian, RNN truyền thống vẫn tồn tại một số hạn chế. Khi chuỗi dữ liệu được trình bày theo một chuỗi có kích thước lớn, quá trình lan truyền ngược sai số qua nhiều bước thời gian có thể làm cho gradient trở nên cực kỳ nhỏ hoặc cực kỳ lớn. Lúc này gradient có xu hướng bị mất đi (vanishing gradient) hoặc bùng nổ gradient (exploding gradient) [?, ?]. Khi gradient bị suy giảm quá mức, mô hình sẽ gặp khó khăn trong việc học và ghi nhớ các phụ thuộc dài hạn giữa các phần tử trong chuỗi. Điều này ảnh hưởng trực tiếp đến hiệu quả của RNN trong các bài toán xử lý văn bản, nơi mà ngữ nghĩa của một từ có thể phụ thuộc vào các từ xuất hiện ở vị trí xa trong câu hoặc đoạn văn. Để khắc phục những hạn chế này, các kiến trúc cải tiến như Bộ nhớ dài-ngắn hạn (Long Short-Term Memory – LSTM) đã được đề xuất nhằm tăng cường khả năng ghi nhớ thông tin dài hạn và cải thiện hiệu suất của mô hình trên dữ liệu tuần tự.

### 1.4.4 Bộ nhớ dài-ngắn hạn (Long Short-Term Memory – LSTM)

Bộ nhớ dài-ngắn hạn (Long Short-Term Memory - LSTM) [?] là một kiến trúc RNN được thiết kế để giải quyết một số hạn chế của RNN trong khả năng ghi nhớ thông tin dài hạn. Trong khi kiến trúc RNN đơn giản sử dụng một trạng thái ẩn đơn giản để truyền thông tin từ một bước thời gian sang một bước thời gian khác, kiến trúc LSTM được trang bị một bộ nhớ tế bào (memory cell) và ba cơ chế cổng kiểm soát, gồm cổng quên, cổng đầu vào, và cổng đầu ra.



Hình 1.3: Minh họa cấu trúc mạng LSTM [?].

Một đặc điểm nổi bật của mạng LSTM là khả năng học và nhớ phụ thuộc lâu dài (long-term dependencies) trong dữ liệu tuần tự. Điều này đặc biệt quan trọng trong những tình huống yêu cầu sự hiểu biết ngữ cảnh lâu dài, chẳng hạn như xử lý ngôn ngữ tự nhiên (NLP), dịch thuật máy, nhận dạng giọng nói, hoặc dự đoán thời gian.

Một cách đặc biệt, bằng việc giải quyết vấn đề gradient biến mất - một vấn đề lớn trong việc huấn luyện mạng lưới hồi tiếp sâu, mạng lưới hồi tiếp lâu dài không chỉ đảm bảo sự ổn định của quá trình học tập, mà đặc biệt phù hợp với những bài toán xử lý văn bản như phân loại tin tức.

Trong cấu trúc của LSTM, mỗi cell được quản lý bởi các cổng chính là: cổng quên (forget gate), cổng vào (input gate), và cổng ra (output gate). Cổng quên được định nghĩa là xác định những thông tin nào cần phải nhớ hay không. Cổng này đóng vai trò quan trọng trong việc loại bỏ những thông tin không cần thiết nữa trong bộ nhớ của LSTM. Từ đó, các thông tin không cần thiết không gây ra nhiễu trong quá trình học nhớ lâu dài. Kế tiếp đó là cổng vào xác định những thông tin nào cần phải học trong bộ nhớ hiện tại. Kết hợp giữa cổng quên và cổng vào là cách mà LSTM học nhớ lâu dài một cách chọn lọc. Cuối cùng là cổng ra xác định những thông tin nào cần phải sử dụng trong bộ nhớ để tạo ra trạng thái ẩn trong thời điểm hiện tại. Từ đó, nó ảnh hưởng đến đầu ra của mô hình. Thông qua các cổng trong cấu trúc của LSTM, nó đã học nhớ lâu dài các phụ thuộc trong chuỗi dữ liệu một cách hiệu quả so với RNN truyền thống.

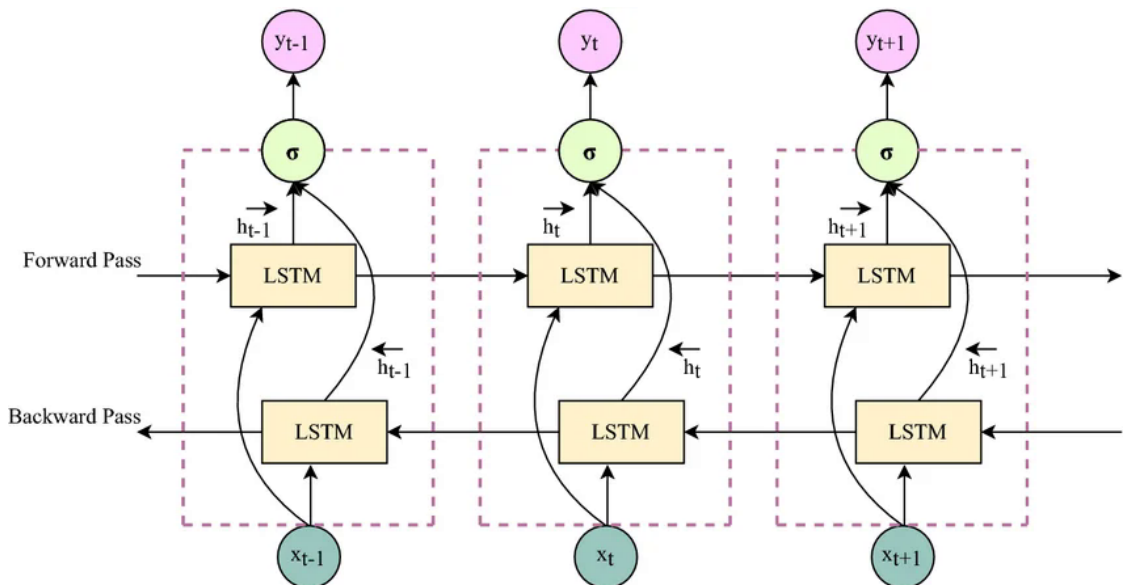
Tuy nhiên, mạng LSTM thông thường chỉ khai thác thông tin trong một chiều, tức là từ quá khứ đến hiện tại. Đối với các bài toán xử lý ngôn ngữ tự nhiên, ngữ nghĩa của một từ không chỉ phụ thuộc vào các từ đứng trước mà có thể phụ thuộc cả về các từ đứng sau trong câu. Vì vậy, kiến trúc mạng LSTM hai chiều (Bidirectional LSTM -

BiLSTM) được đề xuất để mở rộng khả năng biểu diễn của mạng LSTM trong các bài toán phân loại văn bản, như bài toán phát hiện tin giả.

#### 1.4.5 Mạng LSTM hai chiều (Bidirectional LSTM)

Kiến trúc mạng LSTM hai chiều, còn được gọi là Bidirectional LSTM – BiLSTM, là một dạng mở rộng của mạng LSTM. Trong khi LSTM truyền thống chỉ xử lý chuỗi theo một chiều thời gian, thường là từ trái sang phải (từ quá khứ đến hiện tại), thì BiLSTM bao gồm hai mạng LSTM hoạt động song song. Một mạng xử lý chuỗi theo chiều thuận (forward), tức là từ phần tử đầu tiên đến phần tử cuối cùng; mạng còn lại xử lý chuỗi theo chiều ngược (backward), tức là từ cuối về đầu. Cụ thể, với mỗi thời điểm, mạng LSTM thuận sử dụng các phần tử xuất hiện trước đó trong chuỗi dữ liệu để tạo ra một trạng thái ẩn. Ngược lại, mạng LSTM ngược sử dụng các phần tử xuất hiện sau đó để tạo ra một trạng thái ẩn. Hai trạng thái ẩn này sau đó được kết hợp lại để tạo ra một trạng thái cuối cùng để tạo thành biểu diễn ngữ cảnh đầy đủ hơn cho từng phần tử trong chuỗi.

Trong các bài toán xử lý ngôn ngữ tự nhiên, không chỉ có nghĩa của một từ được quyết định bởi từ ngữ trước đó mà còn được quyết định một phần bởi từ ngữ sau đó. Vì vậy, BiLSTM cho phép mô hình nắm bắt được ngữ cảnh của một từ một cách toàn diện hơn so với phương pháp đơn giản của LSTM một chiều. Trong bài toán phát hiện tin giả, việc khai thác ngữ cảnh của một câu và một đoạn văn được cho là có lợi cho mô hình hiểu được nội dung và sắc thái của thông tin, từ đó nâng cao được khả năng phân loại được “thật” và “giả”.



Hình 1.4: Minh họa cấu trúc mạng LSTM hai chiều [?].

### 1.4.6 Cơ chế chú ý (Attention Mechanism)

Cơ chế chú ý (Attention Mechanism) [?] được đưa ra để cải thiện các nhược điểm của các mô hình xử lý chuỗi khi phải nén toàn bộ thông tin của một câu thành một vector. Thay vì mỗi phần tử trong chuỗi đều được tính là có độ quan trọng ngang nhau, cơ chế chú ý cho phép các mô hình tự động học được trọng số cho từng bước trong chuỗi, để tập trung nhiều hơn vào các phần tử quan trọng đối với các tác vụ dự đoán.

Giả sử  $\{h_1, h_2, \dots, h_T\}$  là các trạng thái ẩn thu được từ mô hình LSTM hoặc BiLSTM. Tại mỗi bước thời gian  $t$ , một điểm số  $e_t$  được tính để đo mức độ liên quan của trạng thái ẩn  $h_t$  đối với đầu ra cuối cùng:

$$e_t = \mathbf{v}^\top \tanh(\mathbf{W}_a \overleftrightarrow{\mathbf{h}}_t) \quad (1.1)$$

Các điểm số này sau đó được chuẩn hóa thông qua hàm softmax để tạo thành trọng số chú ý  $\alpha_t$ :

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^T \exp(e_k)} \quad (1.2)$$

Cuối cùng, vector ngữ cảnh  $\mathbf{c}$  được tính bằng tổng có trọng số của các trạng thái ẩn:

$$\mathbf{c} = \sum_{t=1}^T \alpha_t \overleftrightarrow{\mathbf{h}}_t \quad (1.3)$$

Trong đó,  $\alpha_t$  thể hiện mức độ mà từng từ hoặc từng bước trong thời gian đã đóng góp vào biểu diễn cuối cùng của câu. Từ quan trọng sẽ có trọng số cao hơn, còn từ không quan trọng có trọng số nhỏ hơn. Bằng cách này, mô hình có khả năng tập trung vào các thành phần quan trọng của văn bản, chứ không xử lý toàn bộ văn bản một cách đồng đều.

Trong trường hợp của bài toán phát hiện tin giả, cơ chế chú ý cho phép mô hình phát hiện ra từ hoặc cụm từ quan trọng có tác động lớn đến nội dung và ý nghĩa của thông tin, ví dụ, từ thể hiện quan điểm cực đoan, từ thể hiện thông tin gây hiểu lầm, từ thể hiện chi tiết bất thường. Điều này có thể giúp nâng cao khả năng phân biệt được giữa tin “thật” và “giả”.

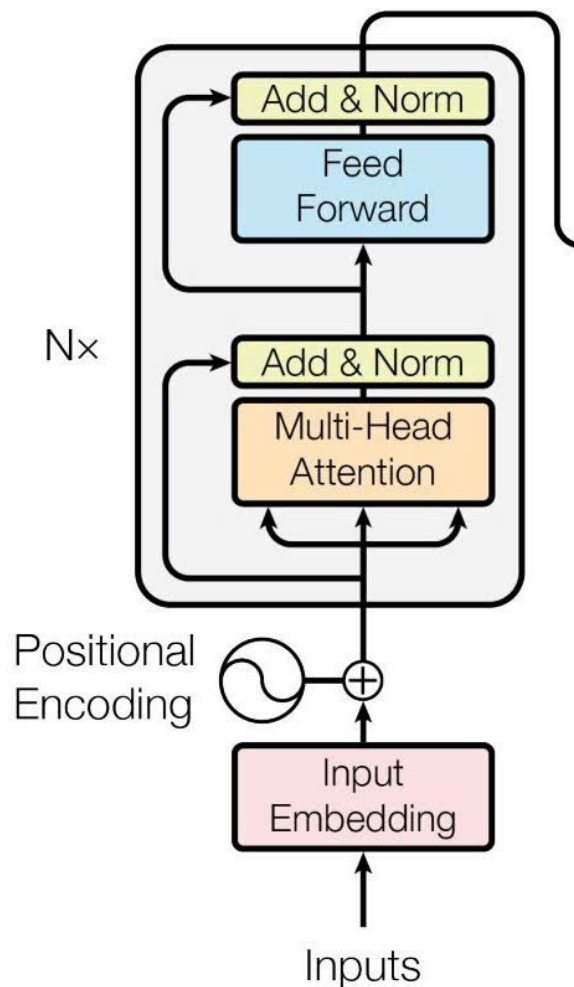
Đáng chú ý, cơ chế tự chú ý (self-attention) là nền tảng cốt lõi của kiến trúc Transformer. Thay vì chỉ tính trọng số dựa trên một trạng thái ẩn duy nhất, self-attention cho phép mỗi phần tử trong chuỗi tương tác trực tiếp với tất cả các phần tử khác, từ đó xây dựng biểu diễn ngữ cảnh toàn cục hiệu quả hơn. Đây chính là cơ sở cho các mô hình ngôn ngữ hiện đại như BERT và PhoBERT được trình bày ở phần

tiếp theo.

### 1.4.7 Mô hình Transformer và PhoBERT

Transformer là một kiến trúc học sâu dựa trên cơ chế tự chú ý (self-attention) [?], được đề xuất nhằm giải quyết các hạn chế của các mô hình tuần tự như RNN và LSTM khi áp dụng trong xử lý phụ thuộc dài hạn. So với các mô hình xử lý dữ liệu theo từng bước thời gian, Transformer cho phép xử lý toàn bộ chuỗi dữ liệu đầu vào một cách đồng thời, từ đó tăng khả năng song song hóa và hiệu quả huấn luyện.

Kiến trúc ban đầu của Transformer bao gồm các thành phần chính là encoder và decoder. Tuy nhiên, trong các mô hình ngôn ngữ tiền huấn luyện như BERT [?] và PhoBERT [?], chỉ sử dụng phần encoder của Transformer để xây dựng biểu diễn ngữ cảnh cho văn bản. Vì vậy, trong phạm vi đề tài nghiên cứu này, chúng tôi tập trung vào kiến trúc của Transformer encoder.



Hình 1.5: Minh họa khối encoder của Transformer [?].

Trong hình trên, có thể thấy được cấu trúc của một khối trong Transformer. Đầu vào của mô hình bao gồm các vector embedding của các từ trong câu và thông tin vị trí. Thông tin vị trí được thêm để đảm bảo thứ tự của các từ trong một câu. Cơ chế attention không xử lý các thông tin một cách tuần tự, vì vậy thông tin vị trí được thêm để đảm bảo thứ tự của các từ trong một câu.

Một khối trong mô hình Transformer được tạo thành từ hai phần. Phần đầu tiên là cơ chế attention multi-head, cho phép các từ trong một câu tương tác trực tiếp với các từ trong cùng một câu để tính toán độ liên quan. Bằng cơ chế attention, các quan hệ ngữ nghĩa trong một văn bản được tính toán mà không bị phụ thuộc vào khoảng cách vị trí. Kết quả của cơ chế attention được đưa đến một bước tính toán (Add & Norm) nhằm ổn định quá trình huấn luyện.

Thành phần thứ hai là mạng truyền thẳng từng vị trí (Positionwise Feed-Forward Network - FFN), thực hiện các phép biến đổi phi tuyến trên từng vector đầu ra của lớp attention. Sau đó, một phép Add & Norm thứ hai được áp dụng. Các khối encoder được xếp chồng nhiều lần để tăng khả năng biểu diễn của mô hình.

Dựa trên khả năng học biểu diễn ngữ cảnh toàn cục thông qua self-attention và khả năng xử lý song song hiệu quả, Transformer đã trở thành nền tảng của các mô hình ngôn ngữ hiện đại. PhoBERT [?] được xây dựng dựa trên kiến trúc của RoBERTa [?], một Transformer encoder nhiều tầng được tiền huấn luyện trên lượng lớn dữ liệu văn bản tiếng Việt.

Trong bài toán phát hiện tin giả, vector tương ứng với token đặc biệt [CLS] được tận dụng như một biểu diễn tổng quan của văn bản và được đi qua một lớp phân loại để dự đoán các nhãn “thật” hay “giả”. Sử dụng biểu diễn ngữ cảnh sâu từ PhoBERT giúp mô hình hiểu rõ hơn nội dung cũng như sắc thái của văn bản, từ đó cải thiện khả năng phân loại.

## 1.5 Kết luận chương

Chương này đã trình bày nền tảng lý thuyết cho bốn mô hình so sánh trong nghiên cứu. Từ LR tuyến tính đến SVM phi tuyến, từ BiLSTM tuần tự đến Transformer song song, mỗi mô hình phản ánh một giai đoạn phát triển của NLP. Chương tiếp theo mô tả bộ dữ liệu và quy trình tiền xử lý.

# Chương 2 Bộ dữ liệu và tiền xử lý

## 2.1 Thu thập và mô tả bộ dữ liệu

### 2.1.1 Nguồn dữ liệu

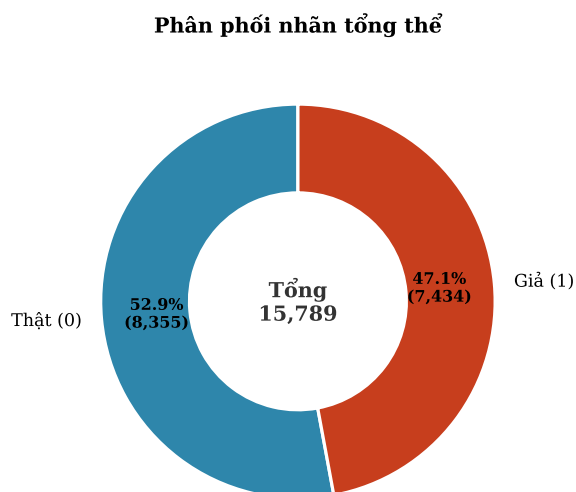
Bộ dữ liệu bao gồm các bài viết tiếng Việt được thu thập từ hai nguồn chính: các bài đăng công khai trên Facebook và một số bài báo cũ trong thời gian diễn ra COVID-19. Mỗi bản ghi chứa tối thiểu hai trường chính:

- **text**: nội dung văn bản của bài viết.
- **label**: nhãn phân loại (0 = Thật, 1 = Giả).

Sau khi thực hiện bước phân đoạn từ (*word segmentation*), dữ liệu được lưu lại và được sử dụng làm đầu vào cho quá trình làm sạch và chia tập.

Tổng số bản ghi ban đầu của bộ dữ liệu là **15,789 mẫu**.

### 2.1.2 Phân phối nhãn



Hình 2.1: Phân phối nhãn theo từng tập dữ liệu và tổng quan toàn bộ 15,789 mẫu

Hình 2.1 minh họa phân phối nhãn tổng thể của toàn bộ tập dữ liệu. Có thể thấy, tỷ lệ tin thật và tin giả khá cân bằng, giúp đảm bảo mô hình không bị thiên lệch về một phía trong quá trình huấn luyện và đánh giá. Sự cân bằng này là yếu tố quan trọng để kết quả thực nghiệm phản ánh đúng hiệu quả của các phương pháp phân loại, đặc biệt trong bối cảnh dữ liệu thực tế thường có sự chênh lệch giữa các lớp.

## 2.2 Tiền xử lý văn bản

### 2.2.1 Làm sạch và lọc dữ liệu

Trước khi thực hiện bước chia tập huấn luyện, dữ liệu được tiến hành làm sạch nhằm loại bỏ nhiễu và hạn chế nguy cơ rò rỉ thông tin. Quy trình làm sạch bao gồm các bước sau:

1. **Loại bỏ văn bản trùng lặp:** Các bản ghi có nội dung `text` giống hệt nhau được loại bỏ, chỉ giữ lại lần xuất hiện đầu tiên.
2. **Loại bỏ văn bản rỗng hoặc giá trị thiếu:** Các bản ghi có trường `text` bị thiếu (NaN) hoặc chuỗi rỗng sau khi loại bỏ khoảng trắng được loại khỏi tập dữ liệu.
3. **Loại bỏ văn bản quá ngắn:** Những văn bản có ít hơn 10 từ bị loại bỏ do không cung cấp đủ thông tin ngữ nghĩa cho bài toán phân loại. Ngưỡng này được cấu hình thông qua tham số `min_word_count = 10`.
4. **Đặt lại chỉ mục (index reset):** Sau khi hoàn tất các bước trên, dữ liệu được đánh lại chỉ mục nhằm đảm bảo tính nhất quán và thuận tiện cho quá trình xử lý tiếp theo.

Sau quá trình làm sạch:

- Số mẫu ban đầu: 15,789
- Số mẫu bị loại bỏ: 1,831
- Số mẫu còn lại: 13,958

Quy trình làm sạch này giúp giảm nhiễu dữ liệu và đảm bảo rằng quá trình huấn luyện cũng như đánh giá mô hình không bị ảnh hưởng bởi các bản ghi trùng lặp hoặc không hợp lệ.



### 2.2.2 Phân đoạn từ bằng VnCoreNLP

Ngôn ngữ tiếng Việt được định nghĩa là một ngôn ngữ đơn lập, trong đó mỗi âm tiết được tách ra với khoảng trắng, mặc dù ranh giới từ không đồng nghĩa với ranh giới âm tiết. Ví dụ, cụm từ “Việt Nam” được định nghĩa là một đơn vị từ gồm hai âm tiết riêng biệt. Điều này tạo ra một thách thức quan trọng đối với xử lý ngôn ngữ tự nhiên vì định nghĩa của mỗi âm tiết là một từ đơn độc lập có thể gây ra sai lệch trong cấu trúc ngữ nghĩa của văn bản. Phân đoạn từ là một tiền xử lý quan trọng để đạt được chính xác trong biểu diễn và trích xuất đặc trưng.

Trong nghiên cứu này, quá trình phân đoạn từ được thư viện `py_vncorenlp` thực hiện, dựa trên một bộ tách từ `RDRSegmenter` của VnCoreNLP [?]. Quá trình này cũng được sử dụng trong phần tiền huấn luyện của mô hình PhoBERT. Sử dụng một bộ phân đoạn giống nhau trong việc xử lý dữ liệu đầu vào và từ vựng của mô hình ngôn ngữ trong phần tiền huấn luyện có khả năng tối đa hóa sự thống nhất, từ đó giảm thiểu độ sai lệch trong không gian biểu diễn và khả năng tổng quát hóa của mô hình.

Quy trình phân đoạn được triển khai theo các bước sau:

- Tự động tải và lưu trữ mô hình VnCoreNLP tại thư mục cục bộ (`.vncorenlp`) nếu chưa tồn tại.
- Thực hiện phân đoạn đối với cả hai trường `title` và `text`.
- Chuẩn hóa văn bản trước khi phân đoạn, bao gồm:
  - Loại bỏ dấu gạch dưới phát sinh từ các lần phân đoạn trước (nếu có).
  - Chuẩn hóa khoảng trắng nhằm đảm bảo tính đồng nhất định dạng.

Sau khi hoàn tất quá trình phân đoạn, các từ đa âm tiết được nối với nhau bằng dấu gạch dưới. Ví dụ:

*Trước:* “Chính phủ Việt Nam công bố chính sách mới”

*Sau:* “Chính\_phủ Việt\_Nam công\_bố chính\_sách mới”

Bước phân đoạn từ đóng vai trò then chốt trong việc cải thiện chất lượng biểu diễn đầu vào, đặc biệt đối với các mô hình dựa trên cơ chế *subword tokenization* như PhoBERT. Nhờ đảm bảo sự tương thích giữa dữ liệu đã xử lý và từ điển tiền huấn luyện, quá trình này góp phần nâng cao độ ổn định và hiệu quả của mô hình trong giai đoạn huấn luyện và suy diễn.

## 2.3 Chia tập dữ liệu

### 2.3.1 Chia phân tầng

Sau khi thực hiện phân đoạn từ, dữ liệu được chia thành ba tập: huấn luyện (train), kiểm định (validation) và kiểm tra (test). Quy trình này bao gồm bước làm sạch dữ liệu trước khi chia tập.

Việc chia tập được thực hiện theo chiến lược phân tầng (*stratified split*) nhằm duy trì phân phối nhãn đồng đều trong cả ba tập. Cấu hình chia tập như sau:

- Train: 70%
- Validation: 15%
- Test: 15%
- `random_state = 42`

Quy trình chia được thực hiện theo hai bước. Đầu tiên, 15% dữ liệu được tách ra làm tập kiểm tra (test set). Sau đó, từ 85% dữ liệu còn lại, tiếp tục tách một phần tương ứng 15% tổng dữ liệu ban đầu (xấp xỉ 12.75% của phần còn lại) để tạo tập kiểm định (validation set).

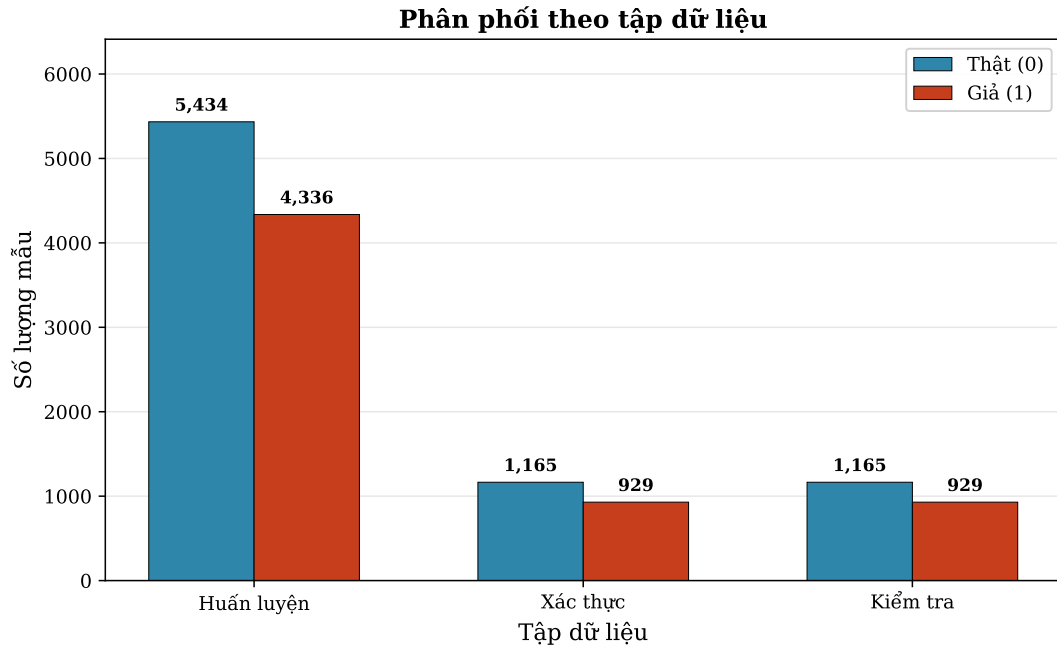
Tham số `stratify = label` đảm bảo rằng phân phối nhãn được bảo toàn giữa các tập:

$$P_{\text{train}}(y) \approx P_{\text{val}}(y) \approx P_{\text{test}}(y)$$

Nhờ đó, sai lệch phân phối nhãn giữa các tập được hạn chế, giúp quá trình đánh giá mô hình phản ánh chính xác hơn khả năng tổng quát hóa.

Kết quả chia tập cuối cùng:

- Train: 9,770 mẫu
- Validation: 2,094 mẫu
- Test: 2,094 mẫu



Hình 2.2: Phân phối nhãn trong các tập dữ liệu huấn luyện, kiểm định và kiểm tra

Bảng 2.1 mô tả thống kê chi tiết sau khi chia tập:

Bảng 2.1: Thống kê tập dữ liệu

| Tập        | Tổng  | Tin thật     | Tin giả      | Độ dài TB |
|------------|-------|--------------|--------------|-----------|
| Huấn luyện | 9,770 | 5,434(55.6%) | 4,336(44.4%) | 90.1      |
| Xác thực   | 2,094 | 1,165(55.6%) | 929(44.4%)   | 91.7      |
| Kiểm tra   | 2,094 | 1,165(55.6%) | 929(44.4%)   | 96.5      |

Bảng 2.1 thể hiện số lượng mẫu, số lượng phân phối nhãn và chiều dài trung bình của văn bản trong các tập dữ liệu. Có thể thấy, các văn bản thật và giả trong các tập huấn luyện, kiểm định và kiểm tra đều có số phần trăm tương ứng là 55.6% và 44.4%, cho thấy chiến lược chia dữ liệu theo phương pháp phân tầng (*stratified split*) có thể bảo toàn số phân phối nhãn ban đầu của dữ liệu. Chiều dài trung bình của văn bản trong các tập dữ liệu kiểm định và kiểm tra (91.7 từ trong tập dữ liệu kiểm định và 96.5 từ trong tập dữ liệu kiểm tra) cao hơn nhẹ so với chiều dài trung bình của văn bản trong tập huấn luyện (90.1 từ), cho thấy đa dạng cấu trúc văn bản trong các dữ liệu. Mặc dù có sự mất cân bằng lớp vừa phải trong các văn bản, số phần trăm này không quá nghiêm trọng và có thể xử lý theo cơ chế trọng số lớp (*class weighting*) trong quá trình huấn luyện để giảm thiểu sai lệch dự đoán.

### 2.3.2 Kiểm tra rò rỉ dữ liệu

Để đảm bảo tính khách quan và độ tin cậy của quá trình đánh giá, nhóm thực hiện bước kiểm tra hiện tượng rò rỉ dữ liệu (*data leakage*) giữa các tập huấn luyện, kiểm định và kiểm tra sau khi thực hiện bước chia dữ liệu.

Cụ thể, toàn bộ nội dung văn bản trong từng tập (Train, Validation, Test) được chuyển đổi thành các tập hợp (*set*) dựa trên giá trị của trường `text`. Sau đó, phép giao được thực hiện giữa từng cặp tập dữ liệu để tìm ra các bản ghi trùng lặp:

- Train và Validation
- Train và Test
- Validation và Test

Kết quả cho thấy:

$$\text{Train} \cap \text{Validation} = \emptyset$$

$$\text{Train} \cap \text{Test} = \emptyset$$

$$\text{Validation} \cap \text{Test} = \emptyset$$

Trước khi tiến hành kiểm tra rò rỉ dữ liệu, cần lưu ý rằng vấn đề này đặc biệt quan trọng trong các bài toán phân loại văn bản. Sự xuất hiện của mẫu trùng lặp giữa tập dữ liệu huấn luyện và tập dữ liệu kiểm tra có thể dẫn đến đánh giá thiên lệch (*biased evaluation*). Trong trường hợp đó, năng suất cao của mô hình có thể đạt được một cách giả tạo nhờ “ghi nhớ” nội dung của tập dữ liệu tập huấn, thay vì thực sự học được đặc trưng có khả năng tổng quát hóa.

## 2.4 Kết luận chương

Chương này đã trình bày quy trình hoàn chỉnh trong việc chuẩn bị dữ liệu cho bài toán phát hiện tin giả tiếng Việt. Các khâu trong quy trình này bao gồm khâu phân đoạn từ, khâu làm sạch dữ liệu, và khâu phân tập dữ liệu bằng chiến lược phân tầng. Dữ liệu được phân đoạn bằng thư viện `RDRSegmenter` để có khả năng tương thích với mô hình `PhoBERT`. Sau đó là khâu lọc bỏ dữ liệu có nội dung văn bản lặp đi lặp lại, rỗng hoặc quá ngắn để có thể tăng chất lượng đầu vào cho bài toán.

Sau đó là khâu phân tập dữ liệu với tỷ lệ 70%–15%–15% cho tập huấn luyện, tập kiểm định, tập kiểm tra của dữ liệu với `random_state = 42`. Trong quá trình phân tập dữ liệu này, quy trình này sẽ thực hiện kiểm tra rò rỉ dữ liệu giữa tập huấn luyện và

tập kiểm định để đảm bảo không tồn tại giao nhau về nội dung văn bản. Quy trình này đảm bảo tính nhất quán, khách quan và khả năng tái lập (*reproducibility*).

## Chương 3 Xác định tin giả

Trong chương này, sau phần trình bày một số lời giới thiệu về bài toán phát hiện tin giả tiếng Việt cũng như các ý tưởng trong việc tiếp cận bài toán này của nhóm, một số phương pháp trích chọn đặc trưng thủ công, cũng như các mô hình học sâu được áp dụng, bao gồm các mô hình truyền thống và các mô hình ngôn ngữ tiền huấn luyện, sẽ được trình bày thông qua lịch sử hình thành và nguyên lý của chúng.

### 3.1 Phương pháp đề xuất

Bài toán phát hiện tin giả được xem là một trong những bài toán quan trọng trong lĩnh vực xử lý ngôn ngữ tự nhiên, giữ vai trò rất quan trọng trong bối cảnh thông tin trực tuyến đang phát triển mạnh mẽ. Sự lan truyền nhanh chóng của các nội dung sai lệch có thể ảnh hưởng tiêu cực đến nhận thức của xã hội. Vì vậy, việc luôn luôn nghiên cứu và hoàn thiện các phương pháp phát hiện tin giả được xem là điều cần thiết.

Cho tới nay, nhiều kỹ thuật trích xuất đặc trưng văn bản đã được các nhà nghiên cứu đề xuất và áp dụng hiệu quả, ví dụ như *Bag-of-Words*, *TF-IDF* [?] hay các kỹ thuật biểu diễn dựa trên word embedding [?]. Ngoài ra, các mô hình học sâu, đặc biệt là các mạng nơ-ron hồi tiếp và các mô hình ngôn ngữ tiền huấn luyện như *Transformer* [?] cũng đã thể hiện được tính vượt trội của mình với những kết quả rất hứa hẹn.

Vì vậy, để có thể lựa chọn phương pháp thích hợp và hiệu quả nhất cho bài toán phát hiện tin giả tiếng Việt, nhóm quyết định thực hiện so sánh hiệu năng của các phương pháp trích chọn đặc trưng thủ công cùng các phương pháp truyền thống và hiện đại như các mô hình học sâu. Đối với các phương pháp trích chọn đặc trưng thủ công, nhóm thực hiện các thuật toán phân lớp như *Logistic Regression*, *SVM*, các mô hình học sâu như *BiLSTM*, *PhoBERT*,...

### 3.2 Lý do lựa chọn các nhóm mô hình

Trong bài toán phân loại văn bản, đặc biệt là phát hiện tin giả, việc lựa chọn mô hình cần cân bằng giữa ba yếu tố chính:

- *Khả năng diễn giải* của mô hình
- *Chi phí tính toán*

- *Năng lực biểu diễn ngữ nghĩa*

Dựa trên các tiêu chí này, nghiên cứu so sánh hai nhóm phương pháp chính:

## 1. Nhóm học máy truyền thống

Bao gồm:

- Logistic Regression
- Support Vector Machine

Đầu vào của các mô hình này là đặc trưng TF-IDF [?]. Hai mô hình có nền tảng lý thuyết vững chắc, hoạt động ổn định trên dữ liệu văn bản có đặc trưng chiều cao và thưa, đồng thời thuận lợi cho việc phân tích đóng góp của đặc trưng và kiểm soát hiện tượng quá khớp [?].

Vì vậy, chúng được sử dụng như các *strong baselines* nhằm đánh giá mức cải thiện thực sự của các phương pháp phức tạp hơn.

## 2. Nhóm học sâu

Bao gồm:

- BiLSTM
- PhoBERT

**BiLSTM** kế thừa khả năng ghi nhớ phụ thuộc dài hạn của LSTM [?]. Khi kết hợp với cơ chế attention, mô hình có thể tập trung vào các thành phần thông tin quan trọng trong chuỗi văn bản [?].

**PhoBERT** là mô hình ngôn ngữ tiền huấn luyện dành riêng cho tiếng Việt, được xây dựng trên nền tảng Transformer/RoBERTa [?, ?, ?]. Mô hình này cho phép khai thác ngữ cảnh hai chiều ở mức sâu và thường đạt hiệu năng cao trong các tác vụ phân loại tiếng Việt.

Việc đặt hai nhóm mô hình trong cùng một khung thực nghiệm giúp trả lời ba câu hỏi nghiên cứu chính:

1. Mức độ cạnh tranh của các mô hình truyền thống khi đặc trưng được thiết kế tốt.
2. Lợi ích biên của học biểu diễn ngữ cảnh sâu so với đặc trưng TF-IDF.
3. Giá trị gia tăng của tiền huấn luyện tiếng Việt (PhoBERT) đối với bài toán phát hiện tin giả trong điều kiện dữ liệu thực tế.

### 3.3 Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế theo ba giai đoạn chính:

1. **Tiền xử lý:** Dữ liệu được làm sạch để loại bỏ các ký tự không mong muốn, đồng thời áp dụng phân đoạn từ cho tiếng Việt. Cuối cùng, dữ liệu được chia thành dữ liệu huấn luyện, kiểm định và kiểm tra với tỷ lệ tương ứng để có được kết quả khách quan trong phần đánh giá.
2. **Trích xuất đặc trưng:** Nhóm áp dụng một số phương pháp biểu diễn văn bản khác nhau cho từng mô hình. Cụ thể, nhóm áp dụng phương pháp *TF-IDF* cho hai mô hình là *Logistic Regression* và *SVM*; phương pháp *Word Embedding* (FastText) cho mô hình *BiLSTM*; phương pháp mã hóa *BPE* (*Byte-Pair Encoding*) cho mô hình *PhoBERT* để khai thác thông tin ngữ cảnh ở mức độ sâu.
3. **Phân loại:** Bốn mô hình là *Logistic Regression*, *SVM*, *BiLSTM*, *PhoBERT* được huấn luyện riêng biệt và đánh giá với các tiêu chí là *Accuracy*, *F1-score*, *ROC-AUC*. Dựa trên kết quả thu được, nhóm so sánh và phân tích được hiệu năng của từng phương pháp.

### 3.4 Trích xuất đặc trưng

#### 3.4.1 TF-IDF cho mô hình học máy truyền thống

##### 3.4.1.1 Lý do lựa chọn TF-IDF

Trong các bài toán phân loại văn bản, TF-IDF có thể được coi như một phương pháp biểu diễn đặc trưng cơ bản nhưng hiệu quả, đặc biệt là khi sử dụng kèm với các mô hình phân loại tuyến tính như Hồi quy Logistic (*Logistic Regression*) và Máy vectơ hỗ trợ (*Support Vector Machine*). Phương pháp này biểu diễn văn bản dưới dạng các vector thưa (*sparse vectors*), trong đó mỗi chiều thể hiện mức độ quan trọng của một từ trong văn bản so với toàn bộ tập dữ liệu.

Cũng như nhiều phương pháp biểu diễn khác, mặc dù các phương pháp biểu diễn hiện đại như nhúng từ (*word embedding*) và các mô hình Ngôn ngữ tiền huấn luyện có thể khai thác hiệu quả ngữ nghĩa sâu hơn, thì TF-IDF vẫn rất được ứng dụng trong nhiều nghiên cứu về phân loại văn bản. Cụ thể hơn, đối với các mô hình tuyến tính, biểu diễn TF-IDF có thể khai thác hiệu quả tín hiệu từ vựng (*lexical cues*) có độ phân biệt cao.



Trong trường hợp bài toán phát hiện tin giả, các cụm từ giật tít, các mẫu ngôn ngữ gây chú ý hoặc các từ khóa đặc trưng thường đóng một vai trò quan trọng trong việc phân biệt tin thật và tin giả. Các tín hiệu này có thể nắm bắt một cách hiệu quả bằng các đặc trưng n-gram của TF-IDF. Vậy là TF-IDF đã có thể kết hợp với Logistic Regression và SVM như một baseline mạnh để so sánh với các phương pháp học sâu trong nghiên cứu này.

#### 3.4.1.2 Cấu hình TF-IDF

Để đảm bảo hiệu quả biểu diễn đặc trưng cho các mô hình học máy truyền thống, cấu hình TF-IDF được lựa chọn dựa trên kết quả *ablation study* (Mục 4.4) nhằm tối ưu hóa hiệu năng phân loại. Cụ thể, các siêu tham số được xác định như sau:

- **Kích thước từ điển:** Số lượng đặc trưng được giới hạn ở 40,000 từ/ngữ phổ biến nhất. Điều này giúp giảm thiểu nhiễu và kiểm soát được sự phức tạp của mô hình.
- **N-gram:** Kết hợp giữa unigram và bigram ( $n\text{-gram} = (1, 2)$ ) nhằm khai thác tối ưu cả thông tin ngữ nghĩa đơn lẻ và thông tin ngữ nghĩa lặp tiếp. Từ đó, nâng cao khả năng nhận diện được ngữ cảnh đặc trưng của tin giả.
- **Tần số tài liệu:** Những từ quá ít ( $\text{min\_df}=5$ ) và quá phổ biến ( $\text{max\_df}=0.95$ ) đều bị loại khỏi danh sách đặc trưng. Như vậy, chỉ giữ lại danh sách đặc trưng có giá trị phân biệt cao.
- **TF sublinear:** Dựa trên phép biến đổi

$$\text{TF} = 1 + \log f_{t,d}$$

nhằm giảm bớt ảnh hưởng của sự lặp lại về mặt ngữ nghĩa. Điều này nhằm ưu tiên sự hiện diện của từ hơn là tần số tuyệt đối.

Việc lựa chọn các tham số này được thực hiện một cách hệ thống nhằm cân bằng giữa khả năng biểu diễn đặc trưng và nguy cơ quá khớp, đồng thời đảm bảo tính tổng quát hóa của mô hình trên các tập dữ liệu kiểm định và kiểm tra.

#### 3.4.2 Nhúng từ (Word Embedding) cho BiLSTM

Để phục vụ cho mô hình BiLSTM, biểu diễn văn bản được thực hiện thông qua kỹ thuật nhúng từ (word embedding) kết hợp giữa embedding tiền huấn luyện và học từ dữ liệu. Từ điển được xây dựng dựa trên toàn bộ tập train, loại bỏ các từ xuất hiện

dưới 2 lần nhằm giảm nhiễu và kiểm soát kích thước không gian từ vựng, kết quả thu được **23,166** từ.

Hai token đặc biệt được bổ sung: <PAD> (index 0) dùng để đệm các chuỗi ngắn và <UNK> (index 1) đại diện cho các từ ngoài từ điển. Mỗi văn bản sau tiền xử lý được chuyển đổi thành một chuỗi số nguyên với độ dài tối đa 128, đảm bảo tính nhất quán về kích thước đầu vào cho mô hình tuần tự.

Các vector embedding có chiều 300 được khởi tạo từ bộ embedding tiền huấn luyện FastText [?] phiên bản tiếng Việt (cc.vi.300.bin). FastText sử dụng thông tin n-gram ký tự để xây dựng biểu diễn từ, cho phép xử lý hiệu quả các từ ngoài từ điển bằng cách tổng hợp vector của các n-gram thành phần. Ma trận embedding tiền huấn luyện được nạp vào lớp embedding của mô hình và tiếp tục được tinh chỉnh trong quá trình huấn luyện thông qua lan truyền ngược, cho phép mô hình điều chỉnh biểu diễn ngữ nghĩa phù hợp với nhiệm vụ phân loại tin giả. Việc kết hợp embedding tiền huấn luyện với fine-tuning giúp tận dụng tri thức ngôn ngữ học được từ ngữ liệu lớn (Common Crawl), đồng thời thích ứng tốt với đặc thù dữ liệu tiếng Việt trong miền bài toán.

### 3.4.3 Tokenization PhoBERT (BPE)

Quá trình mã hóa đầu vào cho PhoBERT sử dụng thuật toán Byte-Pair Encoding (BPE) [?] với từ điển gồm 64.000 subword, cho phép mô hình xử lý hiệu quả các từ ngoài từ điển và hiện tượng đa hình trong tiếng Việt. Mỗi văn bản được tiền xử lý như sau:

- Thêm token đặc biệt [CLS] ở đầu và [SEP] ở cuối chuỗi để đánh dấu ranh giới văn bản.
- Chuỗi token được cắt hoặc đệm về độ dài cố định 256 nhằm đảm bảo tính nhất quán đầu vào.
- Sinh `attention_mask` để phân biệt token thực (1) và token đệm (0), hỗ trợ quá trình attention trong mô hình.

Chiến lược này giúp PhoBERT tận dụng tối đa thông tin ngữ cảnh và đảm bảo khả năng tổng quát hóa trên dữ liệu tiếng Việt đa dạng.

## 3.5 Các mô hình phân loại

### 3.5.1 Các mô hình học máy truyền thống

#### 3.5.1.1 Hồi quy Logistic

Mô hình hồi quy Logistic sử dụng điều chuẩn L2 nhằm hạn chế quá khớp và áp dụng cơ chế cân bằng lớp (`class_weight='balanced'`) để xử lý mất cân bằng nhãn. Các siêu tham số được lựa chọn thông qua GridSearchCV 5-fold, đảm bảo tính khách quan và tổng quát hóa:

- Lưới tìm kiếm:  $C \in \{0.5; 1; 5; 10; 20; 50\}$ , `solver`  $\in \{\text{liblinear}, \text{saga}\}$
- Cấu hình tối ưu:  $C = 10$ , `solver` = `saga`, `max_iter` = 3000

Việc lựa chọn này giúp mô hình đạt hiệu năng tốt nhất trên tập kiểm định mà không bị phụ thuộc vào phân phối nhãn.

#### 3.5.1.2 Máy vectơ hỗ trợ

Mô hình máy vectơ hỗ trợ (SVM) sử dụng LinearSVC với trọng số lớp cân bằng để giảm thiểu ảnh hưởng của mất cân bằng nhãn. Quá trình tối ưu siêu tham số được thực hiện qua GridSearchCV với 5-fold cross-validation:

- Lưới tìm kiếm:  $C \in \{0.1; 0.5; 1; 2; 5; 10\}$
- Cấu hình tối ưu:  $C = 0.5$ , LinearSVC kết hợp hiệu chuẩn xác suất Platt [?] (calibration)

Chiến lược này đảm bảo SVM vừa tận dụng được đặc trưng TF-IDF vừa duy trì khả năng phân biệt tốt giữa các lớp.

### 3.5.2 Các mô hình học sâu

Như đã phân tích ở Mục 3.2, BiLSTM và PhoBERT được lựa chọn lần lượt làm đại diện cho nhóm mô hình tuần tự và nhóm mô hình ngôn ngữ tiền huấn luyện. Phần này trình bày chi tiết kiến trúc và cấu hình huấn luyện của hai mô hình.

### 3.5.2.1 Mạng BiLSTM

Kiến trúc BiLSTM được thiết kế gồm các thành phần chính sau:

1. **Embedding:** Ma trận embedding kích thước ( $V \times 300$ ), cho phép học biểu diễn ngữ nghĩa từ dữ liệu.
2. **BiLSTM 1 tầng:** Mỗi tầng có `hidden_dim=128` cho hai chiều (forward/backward), tổng đầu ra mỗi bước là  $2 \times 128$ .
3. **Soft Attention:** Cơ chế soft attention tính trọng số  $\alpha_t$  cho từng bước thời gian, giúp mô hình tập trung vào các từ/cụm từ quan trọng.
4. **Classifier:** Hai lớp dropout [?] (0.3) xen kẽ với các lớp tuyến tính và ReLU [?], giúp giảm overfitting và tăng khả năng biểu diễn phi tuyến.

Chiến lược huấn luyện sử dụng AdamW [?] ( $\eta = 10^{-3}$ , `weight_decay=10-4`), hàm mất mát CrossEntropy có trọng số lớp để xử lý mất cân bằng, `batch=64`, tối đa 20 epochs với dừng sớm (`patience=5`). Ngoài ra, áp dụng ReduceLROnPlateau (`factor=0.5`, `patience=2`) để điều chỉnh tốc độ học và gradient clipping [?] (`max_norm=1.0`) nhằm đảm bảo ổn định quá trình tối ưu.

### 3.5.2.2 Tinh chỉnh PhoBERT

Kiến trúc PhoBERT fine-tuning gồm hai thành phần chính:

1. **Encoder PhoBERT:** 12 lớp Transformer encoder, đầu ra vector [CLS] 768 chiều đại diện toàn cục cho văn bản.
2. **Classifier Head:** Hai lớp Dropout (0.1) xen kẽ với Linear ( $768 \rightarrow 384 \rightarrow 2$ ) và ReLU [?], giúp tăng khả năng biểu diễn phi tuyến và giảm overfitting.

Chiến lược huấn luyện sử dụng AdamW [?] ( $\eta = 3 \times 10^{-5}$ , `weight_decay=0.01`), Linear Warmup (10%), `batch=16`, tối đa 8 epochs, gradient accumulation steps=4, gradient clipping [?] (`max_norm=1.0`) và dừng sớm (`patience=2`).

## 3.6 Chiến lược xử lý mất cân bằng nhãn

Mặc dù tỷ lệ nhãn trong bộ dữ liệu (55.6% Thật / 44.4% Giả) không quá chênh lệch, sự mất cân bằng vẫn có thể ảnh hưởng đến khả năng phân loại, đặc biệt đối với lớp thiểu số (tin giả). Để giảm thiểu tác động này, nhóm áp dụng hai chiến lược bổ trợ nhau:

- **Chia phân tầng (*stratified split*):** Dữ liệu được chia thành các tập huấn luyện, kiểm định và kiểm tra theo phương pháp phân tầng, đảm bảo tỷ lệ giữa các lớp được bảo toàn đồng đều trong từng tập. Cách tiếp cận này ngăn ngừa tình trạng mô hình bị thiên lệch do sự phân bố nhãn không đều giữa các tập.
- **Trọng số lớp trong hàm mất mát (*class weighting*):** Mỗi lớp  $c$  được gán trọng số nghịch đảo tỷ lệ với tần suất xuất hiện:

$$w_c = \frac{N}{K \cdot N_c}$$

trong đó  $N$  là tổng số mẫu,  $K$  là số lớp và  $N_c$  là số mẫu thuộc lớp  $c$ . Trọng số này được tích hợp trực tiếp vào hàm mất mát CrossEntropy, tăng mức phạt cho các mẫu thuộc lớp thiểu số, từ đó cải thiện recall cho lớp tin giả. Đối với các mô hình truyền thống (LR, SVM), cơ chế tương đương được thực hiện thông qua tham số `class_weight='balanced'` trong scikit-learn [?].

Sự kết hợp hai chiến lược trên giúp đảm bảo quá trình huấn luyện và đánh giá phản ánh đúng thực tế phân phối nhãn, đồng thời giảm thiểu nguy cơ mô hình bỏ sót các trường hợp thuộc lớp thiểu số.

### 3.7 Kết luận chương

Chương này đã trình bày đầy đủ từ trích xuất đặc trưng đến kiến trúc và tham số huấn luyện của bốn mô hình. Thiết kế cho phép so sánh công bằng trên cùng bộ dữ liệu và cùng quy trình đánh giá. Chương sau trình bày kết quả thực nghiệm.

# Chương 4 Kết quả thực nghiệm

## 4.1 Môi trường thực nghiệm

Thực nghiệm được thực hiện trên hệ thống: GPU NVIDIA (CUDA 12.8), RAM 32 GB, Python 3.14.2, PyTorch 2.10, Transformers 4.44, scikit-learn [?] 1.8.

Bảng 4.1: Siêu tham số các mô hình

| Mô hình             | Siêu tham số                                                                                                                             |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Logistic Regression | C=10, solver=saga, class_weight=balanced, max_iter=3000                                                                                  |
| SVM                 | C=0.5, LinearSVC, class_weight=balanced                                                                                                  |
| BiLSTM              | Embedding dim=300, Hidden dim=128<br>Num layers=1, Dropout=0,3<br>Learning rate=0,001, Batch size=64<br>Early stopping patience=5        |
| PhoBERT             | Pre-trained: vinai/phobert-base<br>Max length=256, Weight decay=0,01<br>Learning rate=3e-05, Batch size=16<br>Epochs=8, Warmup ratio=0,1 |

Bảng 4.1 tóm tắt siêu tham số cuối cùng sau tối ưu hoá.

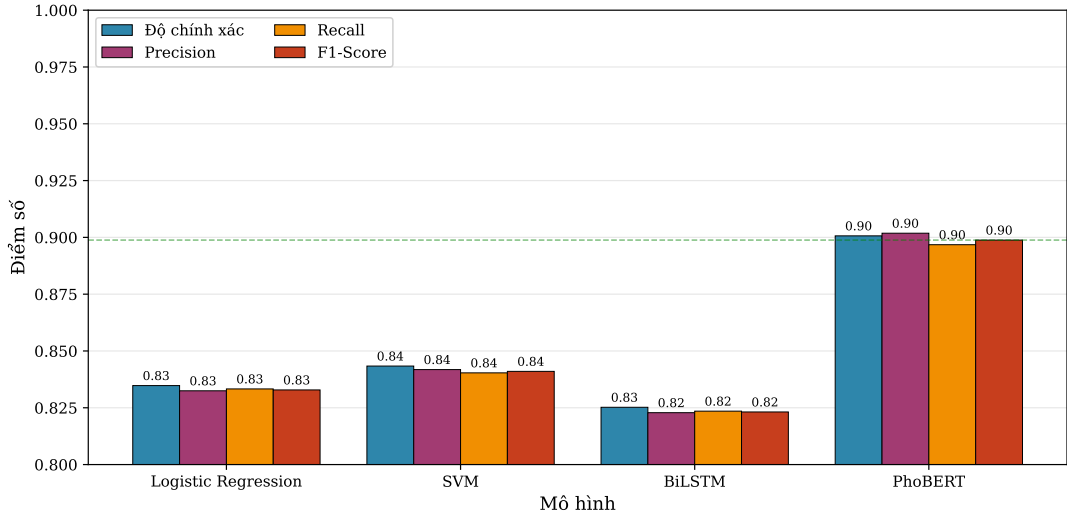
Bảng 4.2: Độ phức tạp và thời gian huấn luyện

| Mô hình             | Loại            | Tham số | Thời gian huấn luyện |
|---------------------|-----------------|---------|----------------------|
| Logistic Regression | ML truyền thống | ~10K    | 1.6 s                |
| SVM                 | ML truyền thống | ~10K    | 3 min                |
| BiLSTM              | Học sâu         | ~500K   | 5 min                |
| PhoBERT             | Transformer     | ~135M   | 1.2 h                |

Bảng 4.2 so sánh số tham số và thời gian huấn luyện giữa các mô hình.

## 4.2 So sánh hiệu năng các mô hình

### 4.2.1 Kết quả tổng thể



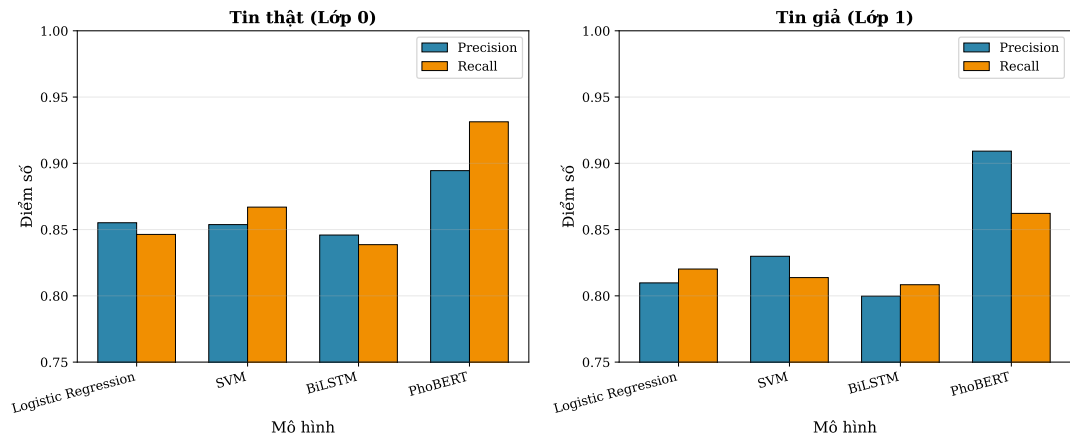
Hình 4.1: So sánh các chỉ số hiệu năng (Accuracy, F1-Score, ROC-AUC) của bốn mô hình trên tập kiểm tra

Bảng 4.3: So sánh hiệu suất các mô hình trên tập kiểm tra

| Mô hình             | Độ chính xác  | Precision     | Recall        | F1-Score      | ROC-AUC       |
|---------------------|---------------|---------------|---------------|---------------|---------------|
| Logistic Regression | 0.8348        | 0.8325        | 0.8333        | 0.8329        | 0.9188        |
| SVM                 | 0.8434        | 0.8418        | 0.8404        | 0.8410        | 0.9190        |
| BiLSTM              | 0.8252        | 0.8228        | 0.8235        | 0.8232        | 0.9048        |
| PhoBERT             | <b>0.9007</b> | <b>0.9018</b> | <b>0.8968</b> | <b>0.8988</b> | <b>0.9495</b> |

Bảng 4.3 trình bày hiệu năng trên tập kiểm tra. PhoBERT đạt kết quả tốt nhất với **F1 = 89.88%** và **AUC = 0.9495**, vượt trội LR (+6.59%), SVM (+5.78%) và BiLSTM (+7.56%). Đáng chú ý, SVM vượt BiLSTM mặc dù kiến trúc đơn giản hơn, cho thấy đặc trưng TF-IDF chất lượng cao cạnh tranh được với học sâu khi dữ liệu giới hạn.

## 4.2.2 Hiệu năng theo từng lớp



Hình 4.2: Precision, Recall và F1-Score theo từng lớp (Thật / Giả) cho bốn mô hình

Bảng 4.4: Chỉ số hiệu suất theo từng lớp

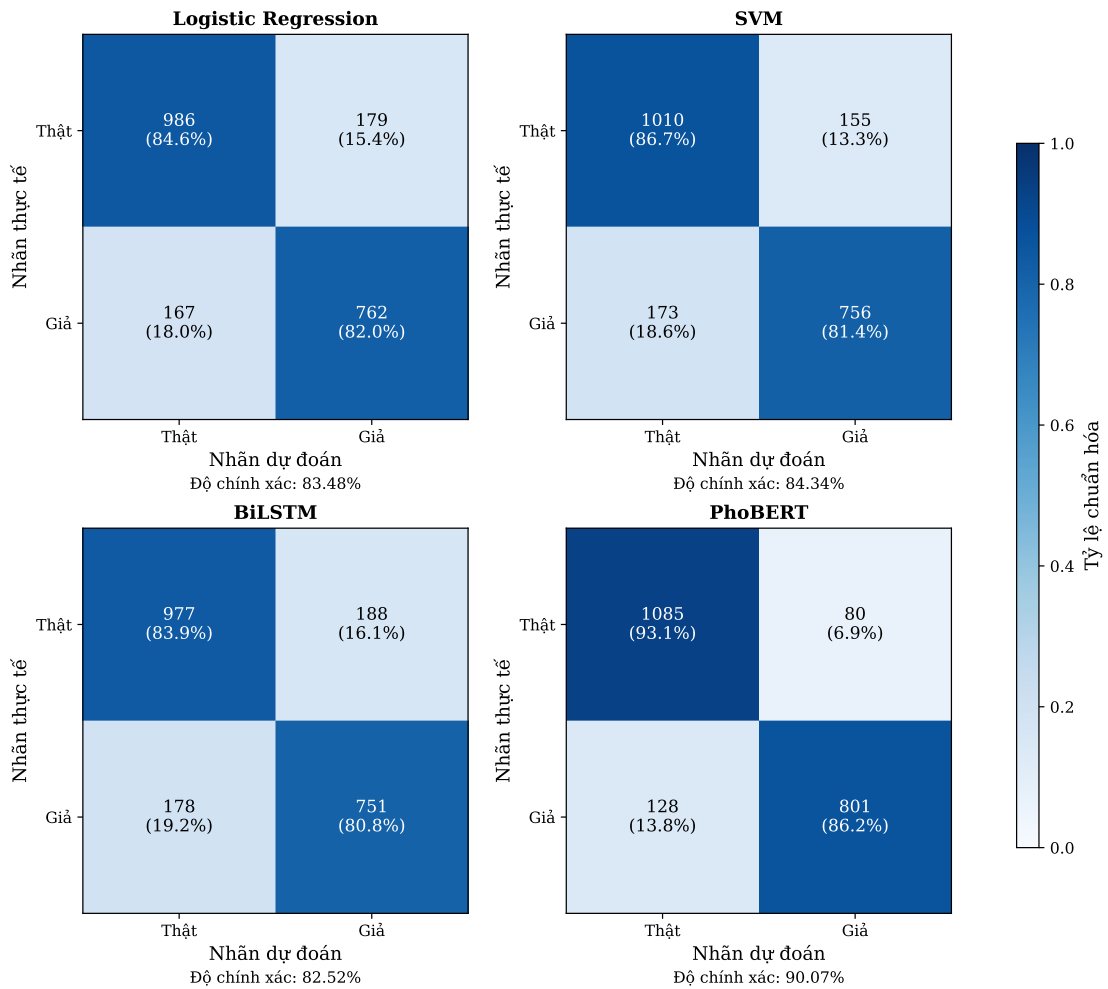
| Mô hình             | Lớp  | Precision | Recall | F1-Score |
|---------------------|------|-----------|--------|----------|
| Logistic Regression | Thật | 0.8552    | 0.8464 | 0.8507   |
|                     | Giả  | 0.8098    | 0.8202 | 0.8150   |
| SVM                 | Thật | 0.8538    | 0.8670 | 0.8603   |
|                     | Giả  | 0.8299    | 0.8138 | 0.8217   |
| BiLSTM              | Thật | 0.8459    | 0.8386 | 0.8422   |
|                     | Giả  | 0.7998    | 0.8084 | 0.8041   |
| PhoBERT             | Thật | 0.8945    | 0.9313 | 0.9125   |
|                     | Giả  | 0.9092    | 0.8622 | 0.8851   |

Từ Bảng 4.4:

- PhoBERT: Recall = 86.22% cho lớp Giả – phát hiện tin giả tốt nhất trong bốn mô hình.
- BiLSTM: Recall = 80.84% cho lớp Giả – bỏ sót nhiều tin giả nhất (nguy hiểm trong thực tế).
- SVM đạt Recall = 81.38% cho lớp Giả, cao hơn LR (82.02%) và BiLSTM.



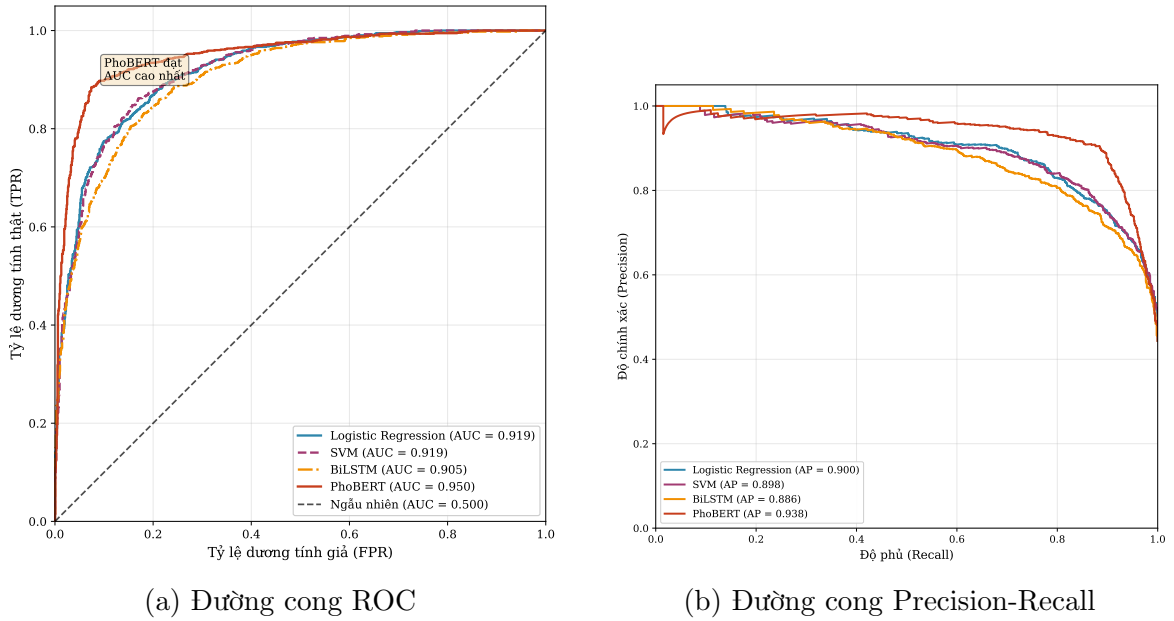
### 4.2.3 Ma trận nhầm lẫn



Hình 4.3: Ma trận nhầm lẫn của bốn mô hình trên tập kiểm tra (2,094 mẫu)

PhoBERT: 1,085 âm tính thật (True Negative – TN), 801 dương tính thật (True Positive – TP), 80 dương tính giả (False Positive – FP), 128 âm tính giả (False Negative – FN) trên 2,094 mẫu test. BiLSTM có nhiều lỗi nhất (366 lỗi, FP=188, FN=178), trong khi PhoBERT ít lỗi nhất (208 lỗi).

#### 4.2.4 Đường cong ROC và Precision-Recall



Hình 4.4: Đường cong ROC và Precision-Recall của bốn mô hình trên tập kiểm tra

PhoBERT ( $AUC = 0.9495$ ) áp đảo so với SVM (0.9190), LR (0.9188) và BiLSTM (0.9048).

#### 4.2.5 Phân tích hiệu chuẩn xác suất (Calibration Analysis)

Bên cạnh các chỉ số phân loại truyền thống, việc đánh giá chất lượng xác suất dự đoán của mô hình cũng đóng vai trò quan trọng trong các ứng dụng thực tế. Một mô hình được coi là có hiệu chuẩn tốt (*well-calibrated*) khi xác suất dự đoán phản ánh đúng tỷ lệ dương tính thực tế: nếu mô hình dự đoán xác suất 80% cho một nhóm mẫu, thì khoảng 80% số mẫu đó phải thực sự thuộc lớp dương.

Điều này đặc biệt quan trọng trong bài toán phát hiện tin giả, vì người dùng hoặc hệ thống downstream cần biết mức độ tin cậy thực sự của mỗi dự đoán để quyết định hành động phù hợp (ví dụ: đánh dấu cảnh báo, chuyển sang kiểm tra thủ công, hoặc tự động loại bỏ).

Nghiên cứu sử dụng ba chỉ số đánh giá hiệu chuẩn [?]:

- **Expected Calibration Error (ECE):** Sai số hiệu chuẩn kỳ vọng, đo mức chênh lệch trung bình có trọng số giữa xác suất dự đoán và tỷ lệ dương tính thực tế trên

các nhóm mẫu (bin):

$$\text{ECE} = \sum_{b=1}^B \frac{|B_b|}{N} \cdot |\text{acc}(B_b) - \text{conf}(B_b)|$$

trong đó  $B_b$  là tập mẫu trong bin thứ  $b$ ,  $\text{acc}(B_b)$  là tỷ lệ dự đoán đúng và  $\text{conf}(B_b)$  là xác suất dự đoán trung bình trong bin đó.

- **Maximum Calibration Error (MCE):** Sai số hiệu chuẩn cực đại, phản ánh bin có độ lệch lớn nhất:

$$\text{MCE} = \max_b |\text{acc}(B_b) - \text{conf}(B_b)|$$

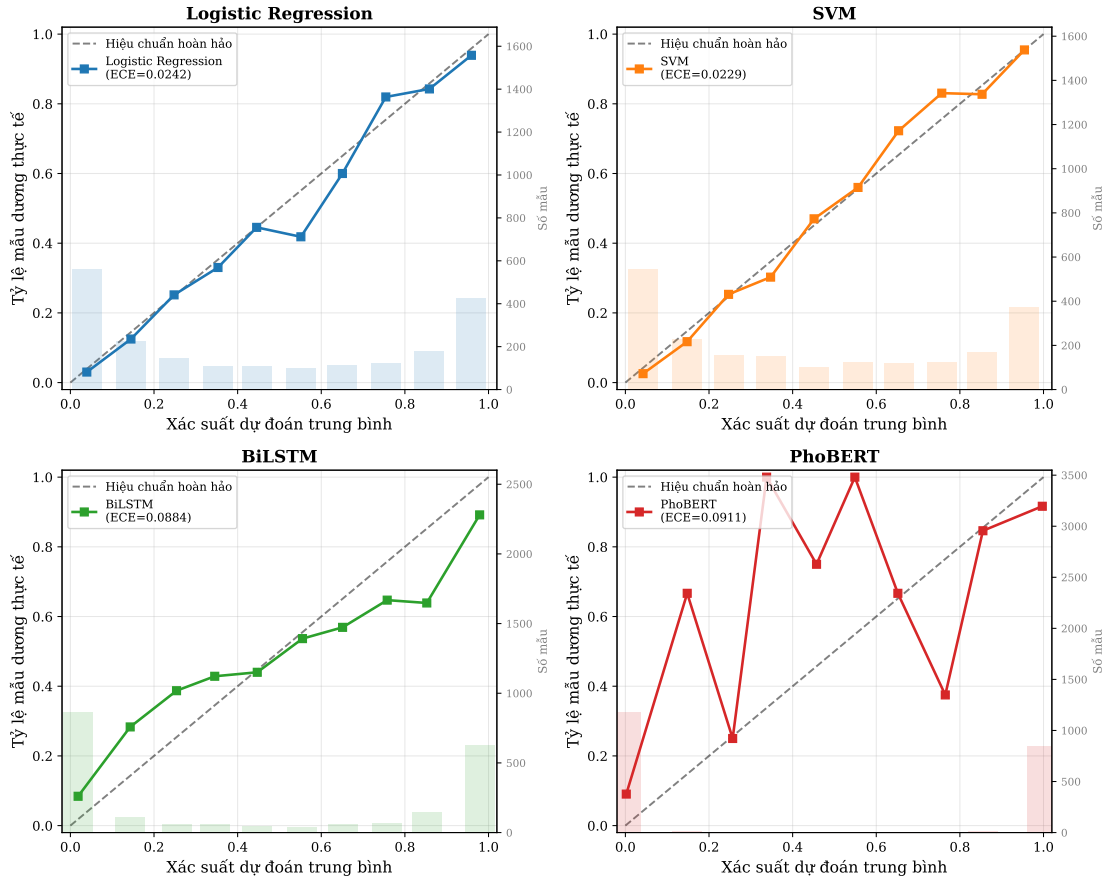
- **Brier Score:** Trung bình bình phương sai số giữa xác suất dự đoán và nhãn thực:

$$\text{BS} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

Brier Score kết hợp cả khả năng phân loại và hiệu chuẩn trong một chỉ số duy nhất.

Cả ba chỉ số đều có giá trị càng thấp càng tốt (0 = hoàn hảo).

Biểu đồ hiệu chuẩn (Reliability Diagram) của bốn mô hình



Hình 4.5: Biểu đồ hiệu chuẩn (Reliability Diagram) của bốn mô hình trên tập kiểm tra. Đường chéo nét đứt biểu diễn hiệu chuẩn hoàn hảo; các điểm vuông biểu diễn tỷ lệ dương tính thực tế trong mỗi bin xác suất.

Bảng 4.5: Chỉ số hiệu chuẩn xác suất của bốn mô hình trên tập kiểm tra

| Mô hình             | ECE ↓         | MCE ↓         | Brier Score ↓ |
|---------------------|---------------|---------------|---------------|
| Logistic Regression | 0.0242        | 0.1329        | 0.1136        |
| SVM                 | <b>0.0229</b> | <b>0.0740</b> | 0.1134        |
| BiLSTM              | 0.0884        | 0.2135        | 0.1348        |
| PhoBERT             | 0.0911        | 0.6622        | <b>0.0925</b> |

Từ Bảng 4.5 và Hình 4.5, có thể rút ra một số quan sát đáng chú ý:

- **Các mô hình học máy truyền thống có hiệu chuẩn tốt nhất:** SVM đạt ECE thấp nhất (0.0229) và MCE thấp nhất (0.0740), theo sau là LR (ECE = 0.0242). Điều này cho thấy xác suất dự đoán của SVM và LR phản ánh khá chính xác tỷ lệ

dương tính thực tế. Kết quả này phù hợp với lý thuyết, vì Logistic Regression được thiết kế để ước lượng xác suất trực tiếp, và SVM sử dụng hiệu chuẩn Platt [?] (CalibratedClassifierCV) trong quá trình huấn luyện.

- **PhoBERT có Brier Score tốt nhất nhưng MCE cao:** PhoBERT đạt Brier Score thấp nhất (0.0925), phản ánh khả năng phân loại vượt trội. Tuy nhiên, MCE rất cao (0.6622) cho thấy mô hình có xu hướng *overconfident* – đưa ra xác suất rất cao (gần 0 hoặc 1) ngay cả khi dự đoán sai. Hiện tượng này là đặc trưng phổ biến của các mô hình Transformer, đặc biệt sau quá trình fine-tuning trên tập dữ liệu nhỏ [?].
- **BiLSTM có hiệu chuẩn kém nhất:** Với ECE = 0.0884 và Brier Score = 0.1348, BiLSTM cho thấy xác suất dự đoán không phản ánh tốt độ chính xác thực tế, nhất quán với hiệu năng phân loại thấp nhất trong bốn mô hình.

**Hàm ý thực tế.** Phân tích hiệu chuẩn cho thấy trong các hệ thống triển khai thực tế, *không nên sử dụng trực tiếp* xác suất thô (raw probability) của PhoBERT làm mức độ tin cậy. Thay vào đó, cần áp dụng kỹ thuật hiệu chuẩn hậu xử lý (*post-hoc calibration*) như Temperature Scaling [?] hoặc Platt Scaling [?] để cải thiện chất lượng xác suất trước khi đưa vào hệ thống ra quyết định. Ngược lại, xác suất từ SVM và LR có thể được sử dụng trực tiếp với độ tin cậy cao hơn.

## 4.3 Kiểm định thống kê

### 4.3.1 Kiểm định McNemar

Kiểm định McNemar [?] được sử dụng để so sánh hiệu năng của hai mô hình phân loại trên cùng một tập kiểm thử, dựa trên các dự đoán theo từng mẫu (paired predictions). Kiểm định này đặc biệt phù hợp trong bối cảnh hai mô hình được đánh giá trên cùng dữ liệu và cần xác định liệu sự khác biệt quan sát được có mang ý nghĩa thống kê hay chỉ do ngẫu nhiên.

Giả thuyết kiểm định được thiết lập như sau:

- $H_0$ : Không tồn tại sự khác biệt có ý nghĩa thống kê giữa xác suất dự đoán đúng của hai mô hình.
- $H_1$ : Tồn tại sự khác biệt có ý nghĩa thống kê giữa xác suất dự đoán đúng của hai mô hình.

Thống kê kiểm định được tính theo công thức:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}$$

Hai đại lượng  $b$  và  $c$  đại diện cho các trường hợp bất đồng (discordant pairs) giữa hai mô hình. Các trường hợp cả hai cùng đúng hoặc cùng sai không ảnh hưởng đến thống kê kiểm định vì không phản ánh sự khác biệt tương đối.

Giá trị  $\chi^2$  càng lớn cho thấy mức độ chênh lệch giữa  $b$  và  $c$  càng cao, tức là sự khác biệt có xu hướng mang tính hệ thống thay vì ngẫu nhiên. Giá trị  $p$ -value được tính dựa trên phân phối Chi-square với 1 bậc tự do. Nếu  $p < \alpha$  (với  $\alpha = 0.05$ ), giả thuyết  $H_0$  bị bác bỏ và có thể kết luận sự khác biệt giữa hai mô hình là có ý nghĩa thống kê.

Hệ số điều chỉnh  $(-1)$  trong tử số là hiệu chỉnh liên tục (*continuity correction*), giúp giảm sai số loại I khi kích thước mẫu không quá lớn.

Do nghiên cứu thực hiện sáu phép so sánh cặp mô hình, việc tiến hành nhiều kiểm định đồng thời có thể làm gia tăng xác suất sai loại I. Để kiểm soát mức sai số toàn cục (*family-wise error rate*), phương pháp hiệu chỉnh Holm–Bonferroni được áp dụng.

Cụ thể, các  $p$ -value được sắp xếp theo thứ tự tăng dần và so sánh tuần tự với các ngưỡng hiệu chỉnh tương ứng, bảo đảm rằng xác suất xuất hiện ít nhất một kết luận sai không vượt quá mức ý nghĩa  $\alpha$  đã lựa chọn.

Với hiệu chỉnh Holm–Bonferroni [?] cho 6 phép so sánh:

Bảng 4.6: Kết quả kiểm định McNemar (sau hiệu chỉnh Holm–Bonferroni)

| Mô hình 1 | Mô hình 2 | $\chi^2$ | $p$ -value | Kết luận         |
|-----------|-----------|----------|------------|------------------|
| LR        | SVM       | 4.52     | 0.0945     | Không có ý nghĩa |
| LR        | BiLSTM    | 1.24     | 0.2645     | Không có ý nghĩa |
| LR        | PhoBERT   | 57.22    | <0.001     | Có ý nghĩa (***) |
| SVM       | BiLSTM    | 4.63     | 0.0945     | Không có ý nghĩa |
| SVM       | PhoBERT   | 45.68    | <0.001     | Có ý nghĩa (***) |
| BiLSTM    | PhoBERT   | 66.62    | <0.001     | Có ý nghĩa (***) |

PhoBERT vượt trội có ý nghĩa thống kê so với cả ba mô hình còn lại ( $p < 0.001$ ). LR, SVM và BiLSTM không khác biệt đáng kể với nhau ( $p \geq 0.05$  sau hiệu chỉnh Holm–Bonferroni) dù kiến trúc khác nhau hoàn toàn.

### 4.3.2 Khoảng tin cậy Bootstrap

Bên cạnh kiểm định McNemar, nghiên cứu tiếp tục sử dụng phương pháp Bootstrap nhằm ước lượng độ ổn định của chỉ số F1-score và đánh giá độ tin cậy của sự khác biệt

giữa các mô hình.

Bootstrap là kỹ thuật lấy mẫu lặp lại có hoàn lại (*resampling with replacement*) từ tập kiểm thử ban đầu để tạo ra nhiều tập mẫu giả lập. Trong nghiên cứu này, quá trình lấy mẫu được thực hiện 10,000 lần. Với mỗi lần lấy mẫu, F1-score được tính lại, từ đó xây dựng phân phối thực nghiệm của chỉ số này.

Khoảng tin cậy 95% (95% Confidence Interval – CI) được xác định từ phân vị 2.5% và 97.5% của phân phối Bootstrap. Khoảng này biểu diễn miền giá trị mà tham số thực (F1-score kỳ vọng trên tổng thể) có khả năng nằm trong đó với mức tin cậy 95%.

Việc sử dụng Bootstrap mang ý nghĩa quan trọng trong đánh giá thực nghiệm vì:

- Không giả định phân phối chuẩn của F1-score.
- Phản ánh mức độ dao động của mô hình khi dữ liệu thay đổi.
- Cho phép so sánh mức độ chồng lấp giữa các khoảng tin cậy, từ đó hỗ trợ xác nhận sự khác biệt có tính hệ thống.

Kết quả khoảng tin cậy 95% cho các mô hình được trình bày trong bảng 4.7.

Bảng 4.7: Khoảng tin cậy 95% Bootstrap [?] cho F1-Score (10,000 lần lấy mẫu lại)

| Mô hình | F1-Score     | 95% CI                  |
|---------|--------------|-------------------------|
| LR      | 83.29        | [0.8165; 0.8487]        |
| SVM     | 84.10        | [0.8253; 0.8564]        |
| BiLSTM  | 82.32        | [0.8063; 0.8391]        |
| PhoBERT | <b>89.88</b> | <b>[0.8853; 0.9114]</b> |

Từ Bảng 4.7 có thể quan sát rằng:

- PhoBERT đạt F1-score trung bình cao nhất (89.88%) với khoảng tin cậy 95% là [0.8853; 0.9114].
- Khoảng tin cậy của PhoBERT không chồng lấp với bất kỳ mô hình nào còn lại.
- Các mô hình LR, SVM và BiLSTM có khoảng tin cậy tương đối gần nhau và tồn tại hiện tượng chồng lấp.

Sự không chồng lấp khoảng tin cậy giữa PhoBERT và các mô hình khác củng cố kết luận từ kiểm định McNemar rằng sự vượt trội của PhoBERT không phải do dao động ngẫu nhiên mà mang tính ổn định thống kê. Ngược lại, sự chồng lấp giữa LR, SVM và

BiLSTM cho thấy sự khác biệt hiệu năng giữa ba mô hình này chưa đủ mạnh để kết luận tồn tại ưu thế rõ ràng.

Như vậy, Bootstrap đóng vai trò bổ sung cho kiểm định McNemar: nếu McNemar kiểm tra sự khác biệt ở mức dự đoán từng mẫu (*instance-level comparison*), thì Bootstrap đánh giá độ ổn định của chỉ số tổng hợp (F1-score) dưới biến thiên dữ liệu (*metric-level stability*). Việc kết hợp cả hai phương pháp giúp tăng độ tin cậy và tính thuyết phục cho kết luận thực nghiệm.

### 4.3.3 Kiểm tra chéo (Cross-Validation)

Bên cạnh việc đánh giá trên tập kiểm thử cố định, nghiên cứu tiếp tục thực hiện kiểm tra chéo phân tầng (*Stratified Cross-Validation*) nhằm đánh giá tính ổn định của mô hình dưới các cách chia dữ liệu khác nhau.

Cụ thể, phương pháp 5-fold cross-validation được áp dụng với 3 seed ngẫu nhiên khác nhau, tạo thành tổng cộng 15 lần huấn luyện và đánh giá. Việc lặp lại trên nhiều seed giúp giảm ảnh hưởng của tính ngẫu nhiên trong quá trình chia dữ liệu và cung cấp ước lượng ổn định hơn về hiệu năng trung bình.

Trong mỗi lần lặp, mô hình được huấn luyện trên 4 folds và kiểm tra trên fold còn lại. Các chỉ số Accuracy và F1-score được ghi nhận cho từng lần chạy, sau đó tính giá trị trung bình và độ lệch chuẩn.

Độ lệch chuẩn ( $\pm$ ) phản ánh mức độ dao động của hiệu năng giữa các lần chia dữ liệu. Giá trị này càng nhỏ cho thấy mô hình càng ổn định và ít phụ thuộc vào cách phân chia tập dữ liệu. Do hạn chế về tài nguyên tính toán và thời gian huấn luyện đáng kể của các mô hình học sâu như BiLSTM và PhoBERT, quy trình kiểm tra chéo 5-fold với 3 seed (tổng 15 lần chạy) chỉ được áp dụng cho các mô hình học máy truyền thống (Logistic Regression và SVM). Việc mở rộng quy trình này cho các mô hình học sâu sẽ đòi hỏi chi phí tính toán rất lớn và vượt quá phạm vi thực nghiệm của nghiên cứu.

Kết quả kiểm tra chéo được trình bày trong Bảng 4.8.

Bảng 4.8: Kết quả kiểm tra chéo 5-fold (3 seeds)

| Mô hình             | Accuracy            | F1-Score            |
|---------------------|---------------------|---------------------|
| Logistic Regression | $0.8301 \pm 0.0087$ | $0.8282 \pm 0.0087$ |
| SVM                 | $0.8490 \pm 0.0087$ | $0.8469 \pm 0.0087$ |

Từ Bảng 4.8 có thể quan sát rằng:

- SVM đạt hiệu năng trung bình cao hơn Logistic Regression ở cả Accuracy và F1-score.



- Độ lệch chuẩn của cả hai mô hình đều nhỏ ( $\leq 0.010$ ), cho thấy hiệu năng ổn định qua các lần chia dữ liệu.
- Mức dao động thấp xác nhận rằng kết quả không phụ thuộc đáng kể vào một cấu hình phân chia cụ thể.

So với kiểm định McNemar và Bootstrap, Cross-Validation bổ sung một góc nhìn khác trong đánh giá thực nghiệm: nếu McNemar kiểm tra sự khác biệt ở mức dự đoán từng mẫu (*instance-level comparison*) và Bootstrap đánh giá độ ổn định của chỉ số trên tập kiểm thử cố định, thì Cross-Validation kiểm tra khả năng tổng quát hóa của mô hình khi dữ liệu huấn luyện thay đổi.

Việc kết hợp ba phương pháp đánh giá (McNemar, Bootstrap và Cross-Validation) giúp đảm bảo rằng kết luận về hiệu năng mô hình không chỉ dựa trên một lần thử nghiệm duy nhất, mà được xác nhận dưới nhiều góc độ thống kê khác nhau.

## 4.4 Nghiên cứu triệt bỏ (Ablation Study)

Ablation Study là phương pháp thực nghiệm nhằm đánh giá mức độ đóng góp của từng thành phần trong một hệ thống bằng cách thay đổi hoặc loại bỏ từng yếu tố riêng lẻ trong khi giữ nguyên các thành phần còn lại. Mục tiêu của phương pháp này không chỉ là tìm cấu hình tốt nhất, mà còn xác định thành phần nào thực sự tạo ra cải thiện hiệu năng.

Trong nghiên cứu này, Ablation Study được thực hiện trên pipeline TF-IDF + Logistic Regression bằng cách lần lượt thay đổi các yếu tố sau:

- Kích thước từ điển
- Cấu hình  $n$ -gram
- Phương pháp phân đoạn từ
- Tham số điều chuẩn  $C$
- Phương pháp chuẩn hóa TF

Mỗi yếu tố được thay đổi độc lập trong khi giữ nguyên các thành phần còn lại, nhằm đảm bảo có thể đánh giá riêng tác động của từng thành phần lên các chỉ số Accuracy và F1-score. Cách tiếp cận này cho phép xác định mức độ nhạy cảm của mô hình đối với từng cấu hình và làm rõ yếu tố nào đóng vai trò quyết định trong việc cải thiện hiệu năng.

Do cấu trúc pipeline TF-IDF + Logistic Regression cho phép tách biệt rõ ràng từng thành phần (kích thước từ điển, cấu hình  $n$ -gram, tham số điều chuẩn  $C$ , phương pháp chuẩn hóa TF), phương pháp ablation study được áp dụng chủ yếu cho mô hình này nhằm phân tích đóng góp riêng lẻ của từng yếu tố.

Đối với các mô hình học sâu như BiLSTM và PhoBERT, biểu diễn đặc trưng và quá trình học được tích hợp chặt chẽ trong kiến trúc mạng, khiến việc loại bỏ hoặc thay đổi từng thành phần riêng lẻ trở nên phức tạp và khó diễn giải. Việc điều chỉnh một thành phần (ví dụ: embedding, số tầng, chiều ẩn hoặc cơ chế attention) thường kéo theo thay đổi trong toàn bộ động học huấn luyện, do đó không đảm bảo tính so sánh độc lập như trong pipeline tuyến tính.

Bên cạnh đó, chi phí tính toán và thời gian huấn luyện lớn cũng hạn chế khả năng thực hiện ablation toàn diện cho các mô hình này trong phạm vi nghiên cứu. Vì vậy, ablation study được tập trung vào mô hình truyền thống nhằm đảm bảo tính minh bạch, khả năng diễn giải và hiệu quả thực nghiệm.

Kết quả thực nghiệm được trình bày trong Bảng 4.9.

Bảng 4.9: Kết quả Ablation Study trên tập kiểm tra

| Thành phần         | Cấu hình        | Accuracy      | F1-Score      |
|--------------------|-----------------|---------------|---------------|
| Kích thước từ điển | 1,000 từ        | 0.7841        | 0.7819        |
|                    | 5,000 từ        | 0.8233        | 0.8214        |
|                    | 10,000 từ       | 0.8324        | 0.8303        |
|                    | 20,000 từ       | 0.8386        | 0.8368        |
|                    | 50,000 từ       | 0.8395        | 0.8378        |
| N-gram             | Unigram only    | 0.8276        | 0.8256        |
|                    | Uni + Bigram    | 0.8429        | 0.8410        |
|                    | Uni+Bi+Trigram  | 0.8415        | 0.8397        |
|                    | Bigram only     | 0.8152        | 0.8120        |
| Phân đoạn từ       | Có phân đoạn    | 0.8429        | 0.8410        |
|                    | Không phân đoạn | 0.8319        | 0.8300        |
| Giá trị $C$ (LR)   | $C = 0.01$      | 0.7713        | 0.7679        |
|                    | $C = 0.1$       | 0.7942        | 0.7914        |
|                    | $C = 1$         | <b>0.8429</b> | <b>0.8410</b> |
|                    | $C = 10$        | 0.8367        | 0.8348        |
|                    | $C = 100$       | 0.8357        | 0.8336        |
| TF Scaling         | Standard TF     | 0.8386        | 0.8367        |
|                    | Sublinear TF    | <b>0.8429</b> | <b>0.8410</b> |

## Phân tích và ý nghĩa

### 1. Kích thước từ điển

Khi tăng kích thước từ điển từ 1,000 lên 10,000 từ, F1-score cải thiện khoảng +4.84%, cho thấy việc mở rộng không gian đặc trưng giúp mô hình khai thác tốt hơn thông tin ngữ nghĩa. Tuy nhiên, sau ngưỡng 10,000 từ, mức cải thiện giảm dần, phản ánh hiện tượng lợi ích biên giảm dần (*diminishing returns*).

Điều này giúp xác định ngưỡng tối ưu giữa độ biểu diễn và độ phức tạp.

### 2. Cấu hình $n$ -gram

Cấu hình Uni + Bigram đạt hiệu năng tốt nhất. Việc bổ sung bigram giúp mô hình nắm bắt quan hệ ngữ cảnh cục bộ mà unigram đơn lẻ không thể biểu diễn đầy đủ. Tuy nhiên, thêm trigram không mang lại cải thiện đáng kể, có thể do hiện tượng thưa dữ liệu (*sparsity*) tăng cao.

Ablation cho thấy bigram là thành phần đóng góp quan trọng.

### 3. Phân đoạn từ

Phân đoạn từ không tạo khác biệt lớn đối với TF-IDF + Logistic Regression ( $\Delta \approx 1.10\%$ ), cho thấy mô hình tuyến tính ít phụ thuộc vào cấu trúc ngữ cảnh sâu. Điều này khác với các mô hình ngữ cảnh như PhoBERT, nơi phân đoạn từ có vai trò quan trọng hơn.

Kết quả giúp làm rõ đặc tính từng nhóm mô hình.

### 4. Tham số điều chuẩn $C$

Trong phép ablation riêng cho thành phần điều chuẩn ở Bảng 4.9, giá trị  $C = 1$  cho hiệu năng tốt nhất.

- $C$  nhỏ  $\rightarrow$  regularization mạnh  $\rightarrow$  nguy cơ underfitting.
- $C$  lớn  $\rightarrow$  regularization yếu  $\rightarrow$  nguy cơ overfitting.

Ablation xác định điểm cân bằng giữa bias và variance trong bối cảnh thí nghiệm đơn biến này. Tuy nhiên, cấu hình LR cuối cùng dùng cho so sánh mô hình vẫn là  $C = 10$  (Mục mô tả Logistic Regression), theo kết quả GridSearchCV trên pipeline hoàn chỉnh.

### 5. Chuẩn hóa TF

Sublinear TF nhỉnh hơn Standard TF (+0.43% F1), cho thấy log-scaling mang lại lợi ích nhẹ trên tập dữ liệu hiện tại.

## Vậy Ablation Study đã giúp cải tiến gì?

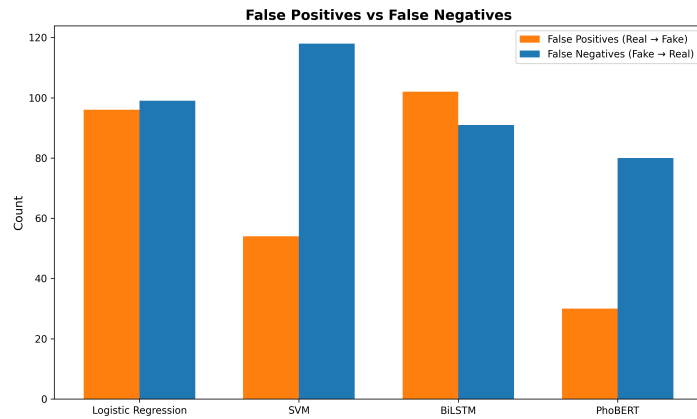
1. Xác định cấu hình tối ưu cho pipeline TF-IDF + Logistic Regression.
2. Loại bỏ các thành phần không đóng góp đáng kể.
3. Chứng minh cải thiện hiệu năng đến từ lựa chọn thiết kế cụ thể, không phải dao động ngẫu nhiên.
4. Cung cấp cơ sở thực nghiệm cho cấu hình sử dụng trong so sánh mô hình ở Mục 4.2.

Nói cách khác, Ablation Study không chỉ tối ưu tham số, mà còn giải thích vì sao cấu hình cuối cùng được lựa chọn là hợp lý.

## 4.5 Phân tích lỗi

### 4.5.1 Thống kê lỗi tổng quan

#### 4.5.1.1 So sánh FP và FN



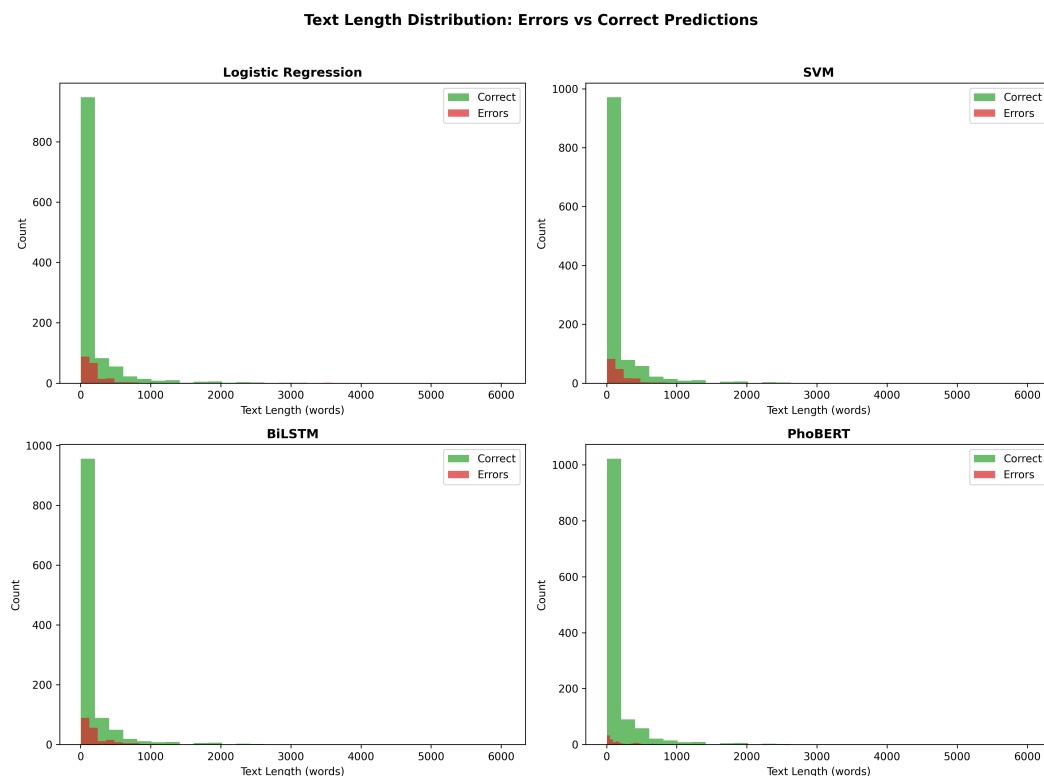
Hình 4.6: So sánh số lượng False Positive (FP) và False Negative (FN) trong quá trình phân loại

Trong Hình 4.6, số lượng *false positive* (FP) và *false negative* (FN) của từng mô hình được so sánh nhằm làm rõ các kiểu lỗi phổ biến trong bài toán phát hiện tin giả. Có thể quan sát rằng các mô hình khác nhau thể hiện xu hướng lỗi khác nhau. Logistic Regression tạo ra số lượng FP và FN khá cân bằng, trong khi SVM có xu hướng tạo ra nhiều FN hơn FP. Điều này cho thấy SVM thường bỏ sót một số trường hợp tin giả, đặc biệt khi các mẫu này không chứa các từ khóa đặc trưng rõ ràng.

Đối với BiLSTM, số lượng lỗi tổng thể cao hơn so với các mô hình còn lại, với cả FP và FN đều ở mức tương đối lớn. Điều này cho thấy mô hình tuần tự này chưa khai thác được đầy đủ thông tin ngữ nghĩa trong văn bản, có thể do kích thước dữ liệu huấn luyện chưa đủ lớn để học các biểu diễn ngữ cảnh phức tạp. Ngược lại, PhoBERT đạt số lượng lỗi thấp nhất trong bốn mô hình, phản ánh khả năng khai thác ngữ cảnh tốt hơn nhờ được tiền huấn luyện trên lượng lớn dữ liệu tiếng Việt.

Phân tích FP và FN cũng cho thấy một số đặc điểm của bài toán. Các FP thường xuất hiện ở những bài viết có tiêu đề mang tính giật gân hoặc chứa các từ khóa thường xuất hiện trong tin giả, khiến mô hình nhầm lẫn và dự đoán sai. Trong khi đó, các FN chủ yếu xảy ra ở những tin giả được viết theo phong cách trung lập hoặc có cấu trúc tương tự các bài báo chính thống, khiến mô hình khó phân biệt.

### 4.5.1.2 Phân phối lỗi theo độ dài văn bản



Hình 4.7: Phân phối lỗi phân loại theo độ dài văn bản

Hình 4.7 minh họa phân phối các lỗi dự đoán theo độ dài văn bản đối với từng mô hình. Có thể thấy rằng phần lớn các lỗi xảy ra ở các văn bản có độ dài tương đối ngắn. Những văn bản ngắn thường thiếu ngữ cảnh và thông tin bổ sung cần thiết để mô hình có thể xác định chính xác tính xác thực của nội dung. Do đó, cả các mô hình học máy truyền thống và mô hình học sâu đều gặp khó khăn khi xử lý các mẫu này.

Ngược lại, khi độ dài văn bản tăng lên, tỷ lệ lỗi có xu hướng giảm. Các văn bản dài thường chứa nhiều thông tin ngữ nghĩa hơn, giúp mô hình nhận diện các mẫu đặc trưng liên quan đến tin thật hoặc tin giả một cách rõ ràng hơn. Xu hướng này đặc biệt rõ rệt đối với PhoBERT, khi mô hình tận dụng tốt hơn thông tin ngữ cảnh dài nhờ kiến trúc Transformer.

Phân tích này cho thấy độ dài văn bản là một yếu tố ảnh hưởng đáng kể đến hiệu năng của mô hình. Trong các ứng dụng thực tế, các bài viết ngắn hoặc tiêu đề đơn lẻ có thể cần được xử lý bằng các kỹ thuật bổ sung, chẳng hạn như kết hợp thông tin ngữ cảnh từ nguồn dữ liệu khác hoặc sử dụng các đặc trưng bổ sung ngoài nội dung văn bản.

#### 4.5.1.3 Hàm ý cải tiến mô hình

Bảng 4.10: Phân tích lỗi trên tập kiểm tra

| Mô hình             | Tổng lỗi | Tỉ lệ lỗi | FP  | FN  |
|---------------------|----------|-----------|-----|-----|
| Logistic Regression | 346      | 16.52%    | 179 | 167 |
| SVM                 | 328      | 15.66%    | 155 | 173 |
| BiLSTM              | 366      | 17.48%    | 188 | 178 |
| PhoBERT             | 208      | 9.93%     | 80  | 128 |

Bảng 4.10 tổng hợp số lượng lỗi và phân bố FP/FN của từng mô hình trên tập kiểm tra.

Từ các kết quả phân tích lỗi trên, có thể rút ra một số hướng cải tiến tiềm năng cho hệ thống phát hiện tin giả. Thứ nhất, việc kết hợp các đặc trưng ngữ nghĩa sâu với các đặc trưng bề mặt như từ khóa hoặc cụm từ đặc trưng có thể giúp giảm các trường hợp *false positive* trong các mô hình học máy truyền thống. Thứ hai, các mô hình học sâu có thể được cải thiện bằng cách bổ sung thêm dữ liệu huấn luyện hoặc áp dụng các chiến lược tăng cường dữ liệu nhằm giúp mô hình học tốt hơn các mẫu tin giả được viết theo phong cách trung lập.

Ngoài ra, phân tích cũng cho thấy các văn bản ngắn là một nguồn gây lỗi phổ biến. Điều này gợi ý rằng các phương pháp khai thác ngữ cảnh mở rộng, chẳng hạn như kết hợp tiêu đề với nội dung bài viết hoặc tích hợp thông tin từ nguồn bên ngoài, có thể giúp cải thiện khả năng dự đoán đối với các trường hợp khó.

#### 4.5.1.4 Phân tích các trường hợp khó (Hard Examples)

Mặc dù các phân tích trên đã chỉ ra một số nguyên nhân phổ biến gây ra lỗi phân loại, kết quả thực nghiệm cho thấy vẫn tồn tại một nhóm trường hợp đặc biệt khó mà tất cả các mô hình đều thất bại. Những trường hợp này phản ánh các giới hạn hiện tại của các phương pháp chỉ dựa trên nội dung văn bản và cung cấp thêm góc nhìn sâu hơn về những thách thức của bài toán phát hiện tin giả.

Trong quá trình phân tích lỗi, nhóm phát hiện **73 mẫu khó** (*hard examples*) mà tất cả bốn mô hình đều dự đoán sai. Việc kiểm tra thủ công nội dung của các mẫu này cho thấy chúng thường thuộc một số dạng đặc trưng, phản ánh những thách thức phổ biến trong bài toán phát hiện tin giả. Cụ thể, các trường hợp này có thể được chia thành ba nhóm chính: *semantic reasoning*, *knowledge verification*, và *dataset ambiguity*.

Trước hết, một số mẫu yêu cầu suy luận ngữ nghĩa (*semantic reasoning*) để xác định mối quan hệ giữa hai câu. Trong các trường hợp này, hai văn bản có thể không

liên quan về mặt nội dung nhưng lại có phong cách viết tương tự, khiến mô hình khó phân biệt. Ví dụ, trong một mẫu dữ liệu, câu “Uống đủ nước mỗi ngày hỗ trợ quá trình trao đổi chất của cơ thể.” được ghép với câu “Ty phú Phạm Nhật Vượng đã quyết định tặng 1000 xe VinFast VF9 cho các y bác sĩ chống dịch Covid-19.” Hai câu này không có quan hệ ngữ nghĩa trực tiếp nhưng đều mang phong cách thông tin trung lập giống các bản tin, dẫn đến việc mô hình nhầm lẫn khi phân loại.

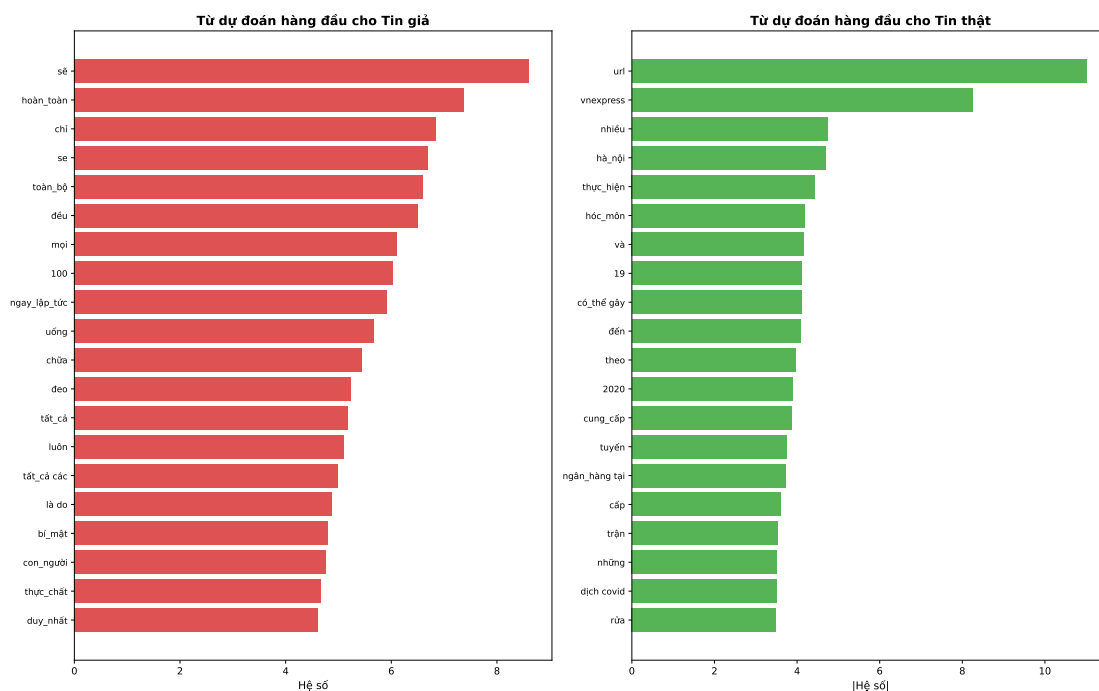
Thứ hai, một số trường hợp thuộc nhóm *knowledge verification*, trong đó việc xác định tính đúng sai của thông tin đòi hỏi kiến thức thực tế ngoài văn bản. Ví dụ, câu “Máy tính cá nhân đầu tiên được bán ra thị trường vào thế kỷ 18.” là một khẳng định sai về mặt lịch sử công nghệ. Tuy nhiên, nếu chỉ dựa vào đặc trưng ngôn ngữ trong văn bản, mô hình khó có thể nhận biết được tính sai lệch của thông tin này. Điều này cho thấy các mô hình hiện tại chủ yếu học các mẫu ngôn ngữ bề mặt và chưa thể thực hiện kiểm chứng tri thức.

Cuối cùng, một số lỗi xuất phát từ *dataset ambiguity*, tức là những trường hợp mà nội dung văn bản không cung cấp đủ ngữ cảnh để xác định nhãn một cách rõ ràng. Chẳng hạn, câu “Hà Nội: học sinh từ cấp 2 trở lên dự kiến đi học trở lại từ 4/5.” có hình thức giống một bản tin chính thống và không chứa dấu hiệu rõ ràng của thông tin sai lệch. Trong các trường hợp như vậy, việc phân loại có thể phụ thuộc vào bối cảnh thời gian hoặc nguồn tin, điều mà các mô hình chỉ dựa trên văn bản không thể khai thác.

Những quan sát này cho thấy các lỗi còn lại không chỉ xuất phát từ hạn chế của mô hình mà còn phản ánh độ phức tạp của bài toán phát hiện tin giả. Các hệ thống trong tương lai có thể được cải thiện bằng cách tích hợp thêm nguồn tri thức bên ngoài hoặc các cơ chế suy luận ngữ nghĩa sâu hơn.



## 4.5.2 Phân tích khả năng giải thích

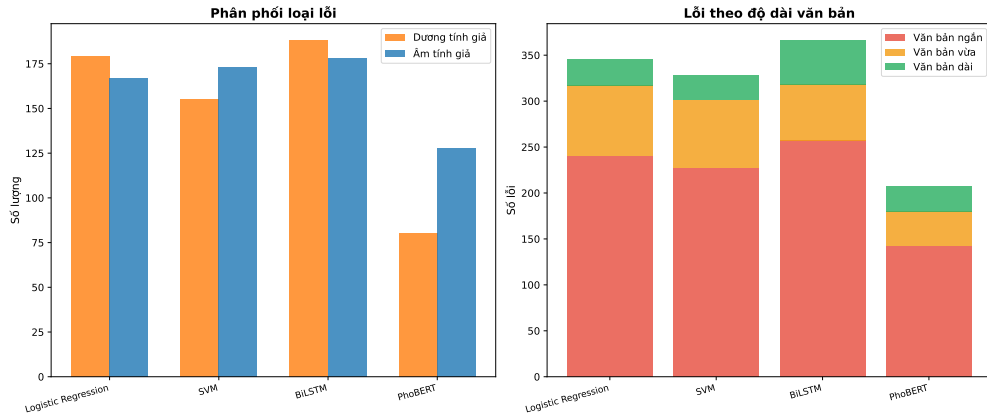


Hình 4.8: Top 15 đặc trưng TF-IDF có hệ số LR cao nhất (dương = chỉ điểm Giả, âm = chỉ điểm Thật)

Hình 4.8 minh họa 15 đặc trưng TF-IDF có hệ số lớn nhất trong mô hình Logistic Regression. Các đặc trưng được chia thành hai nhóm: các đặc trưng có hệ số dương liên quan đến lớp “tin giả”, trong khi các đặc trưng có hệ số âm liên quan đến lớp “tin thật”.

Có thể quan sát rằng nhiều từ khóa liên quan đến tin giả thường mang tính giật gân hoặc gây chú ý, ví dụ các cụm từ mang sắc thái khẳng định mạnh hoặc gây tò mò. Ngược lại, các từ khóa liên quan đến tin thật thường xuất hiện trong văn phong báo chí chính thống, chẳng hạn như các từ liên quan đến cơ quan nhà nước, tổ chức hoặc thuật ngữ chính sách.

Kết quả này cho thấy mô hình Logistic Regression không chỉ học các đặc trưng tần suất đơn thuần mà còn khai thác được những mẫu ngôn ngữ đặc trưng của từng loại tin. Điều này giúp tăng tính minh bạch của mô hình, vì các quyết định phân loại có thể được giải thích trực tiếp thông qua các trọng số đặc trưng.



Hình 4.9: Phân loại lỗi theo độ dài văn bản và mức độ tin cậy dự đoán

Hình 4.9 trình bày phân tích lỗi theo độ dài văn bản và mức độ tin cậy của dự đoán. Ở biểu đồ bên trái, có thể quan sát rằng số lượng lỗi cao hơn đối với các văn bản ngắn. Điều này có thể được giải thích bởi việc các văn bản ngắn thường thiếu ngữ cảnh và chứa ít đặc trưng TF-IDF hơn, khiến mô hình khó phân biệt giữa tin thật và tin giả.

Biểu đồ bên phải cho thấy phần lớn các lỗi xảy ra ở các dự đoán có mức độ tin cậy thấp hoặc trung bình. Trong khi đó, các dự đoán có độ tin cậy cao hiếm khi bị sai. Kết quả này cho thấy xác suất dự đoán của mô hình có tương quan tốt với độ chính xác thực tế.

Phân tích này gợi ý rằng trong các ứng dụng thực tế, những dự đoán có độ tin cậy thấp có thể được chuyển sang bước kiểm tra thủ công hoặc kết hợp với các hệ thống fact-checking để tăng độ tin cậy tổng thể của hệ thống.

## 4.6 Kết luận chương

Thực nghiệm toàn diện xác nhận PhoBERT là mô hình tốt nhất ( $F1 = 89.88\%$ ) và vượt trội có ý nghĩa thống kê. Tiền huấn luyện trên ngữ liệu tiếng Việt là yếu tố quyết định, không phải kiến trúc phức tạp. SVM vượt BiLSTM là minh chứng rõ ràng cho vai trò của đặc trưng chất lượng trong học máy truyền thống.